



VRIJE
UNIVERSITEIT
BRUSSEL



Graduation thesis submitted in partial fulfilment of the requirements for the
degree of de Ingenieurswetenschappen: Computerwetenschappen

DOES DISENTANGLEMENT ACTUALLY IMPROVE ROBUSTNESS IN MULTI-VIEW GENERATIVE LEARNING?

Master Thesis

Niels Koops

August 2025

Promotor: prof. dr. Lynn Houthuys

sciences and bioengineering sciences



VRIJE
UNIVERSITEIT
BRUSSEL



Proefschrift ingediend met het oog op het behalen van de graad van Master of
Science in de Ingenieurswetenschappen: Computerwetenschappen

DOES DISENTANGLEMENT ACTUALLY IMPROVE ROBUSTNESS IN MULTI-VIEW GENERATIVE LEARNING?

Master Thesis

Niels Koops

August 2025

Promotor: prof. dr. Lynn Houthuys

wetenschappen en bio-ingenieurswetenschappen

Abstract

The robustness of machine learning models against adversarial attacks has been studied extensively in classification, but far less in generative modeling. This gap is significant, as generative models such as Variational Autoencoders (VAEs) and their variants are widely used for disentanglement, interpretability, and downstream tasks. At the same time, the assumed link between disentanglement and adversarial robustness remains largely untested, particularly in multi-view generative models.

This thesis systematically investigates whether disentanglement contributes to robustness in generative representation learning. We evaluate the adversarial vulnerability of VAEs, β -VAEs, β -TC-VAEs, and Generative Relational Kernel Machines (Gen-RKMs) under latent-space attacks. To this end, we design attacks that optimize perturbations directly in the latent space and assess their effectiveness on the MNIST dataset. Furthermore, we extend the Gen-RKM framework by incorporating cross-view consistency into the loss function, thereby introducing a novel defense mechanism tailored to multi-view generative models.

Our results show that disentanglement alone does not guarantee robustness: VAEs and their variants exhibit attack success rates between 87–89%, comparable to each other despite differences in latent structure. In contrast, Gen-RKM without defense is highly vulnerable (96.4% success), but enforcing cross-view consistency reduces the attack success rate to 77.3%, demonstrating a substantial robustness gain. These findings highlight that robustness in generative models is not an automatic byproduct of disentanglement, but can be enhanced by leveraging multi-view structure.

Overall, this work highlights the potential of combining disentanglement with multi-view consistency as a promising path toward generative models that are both interpretable and resilient, thereby advancing the reliable deployment of machine learning in sensitive domains.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	Problem Statement	1
1.3	Research Question and Objectives	1
1.4	Approach	2
1.5	Contributions	2
1.6	Structure of the Thesis	2
2	Background	3
2.1	Generative Learning	3
2.2	Multi-View Learning	3
2.3	Disentanglement	3
2.4	Robustness in Machine Learning	4
2.4.1	Noise	4
2.4.2	Outliers	5
2.4.3	Adversarial Attacks	5
3	Literature review	5
3.1	Disentanglement	6
3.1.1	Definition	6
3.1.2	Measuring Disentanglement	7
3.2	Adversarial robustness	7
3.2.1	Threat models	8
3.2.2	Attacking scenario	8
3.2.3	Threat model assumptions	9
3.2.4	Generating the adversarial example	10
3.2.5	Solving the Optimization Problem	11
3.3	The Machine Learning models	11
3.3.1	Variational Autoencoders (VAEs)	11
3.3.2	β -VAE	12
3.3.3	β -TCVAE	12
3.3.4	Generative Restricted Kernel Machines (Gen-RKM)	12
3.3.5	LeNet	13
4	Methodology	14
4.1	Overview	14
4.2	Datasets	14
4.2.1	MNIST	14
4.3	Model Architectures	14
4.3.1	The Autoencoders (VAE, β -VAE, β -TCVAE)	14
4.3.2	Generative Restricted Kernel Machines (Gen-RKM)	15
4.3.3	LeNet Classifier	15
4.4	Disentanglement and Latent Space	16
4.4.1	Measuring Disentanglement	16
4.4.2	Latent Dimensionality and Generating Factors	16
4.5	Latent Attack Framework	16
4.5.1	General Attack Definition & Pseudocode	16
4.5.2	Adaptation for Autoencoders	18

4.5.3	Adaptation for Gen-RKM	19
4.5.4	Attack Extensions	19
4.6	Evaluation Metrics	21
4.6.1	Adversarial Reconstruction Success.	21
4.6.2	Mean Squared Error (MSE).	21
4.6.3	Adversarial Effort (Iterations-to-Success)	22
4.6.4	Latent Space Alignment (Loss)	22
4.7	Robustness Evaluation Framework	23
4.7.1	Multi-View Robustness Evaluation (Gen-RKM Extension)	23
5	Experiments	24
5.1	MNIST Source and Target Image Selection	24
5.2	Experiment 1: Robustness to Latent-Space Adversarial Attacks	25
5.2.1	Setup	25
5.2.2	Results	26
5.3	Experiment 2: Cross-view consistency for multi-view latent targeting of Gen-RKM	26
5.3.1	Setup	26
5.3.2	Results	28
5.4	Discussion	30
6	Results and Discussion	31
6.1	Experiment 1: Disentanglement vs. Robustness	31
6.2	Experiment 2: Leveraging Multi-View Representations as a Defense	31
6.3	Discussion of Loss Balancing and Trade-offs	32
6.4	Linking back to the thesis question	32
6.5	Broader implications	32
7	Conclusion	33
A	LeNet Classification Metrics	34
A.1	Validation Accuracy and loss per Epoch	34
A.2	Classification Report	34
B	Validation Loss of the Autoencoders	35
B.1	VAE	35
B.2	β -VAE	35
B.3	β -TC-VAE	36
C	Additional Information on Experiment 1	36

1 Introduction

1.1 Context and Motivation

Machine learning models are increasingly deployed in sensitive applications, ranging from autonomous driving to medical diagnosis. In such contexts, the *robustness* of learned representations is as important as their accuracy. Research on adversarial robustness has historically focused mostly on *discriminative models* such as classifiers, where small, carefully chosen perturbations can cause severe misclassifications (Dalvi et al. 2004)(Chakraborty et al. 2021). In contrast, the robustness of *generative models*—which are increasingly used for representation learning, disentanglement, and downstream tasks—has received much less systematic attention. This is an important gap, since generative models expose structured latent spaces that may themselves be vulnerable to targeted perturbations.

Generative models such as Variational Autoencoders (VAEs) and their variants provide structured latent representations that enable disentanglement, interpretability, and flexible downstream applications. Yet the same structured latent space that facilitates interpretability may also create vulnerabilities, raising the question of whether disentangled representations are in fact more robust, or more fragile, under adversarial influence.

1.2 Problem Statement

Disentanglement is often assumed to improve robustness by imposing structure on latent spaces (Alemi et al. 2016), but this assumption remains largely untested in adversarial settings. Existing research has primarily analyzed disentanglement from the perspective of interpretability and transferability (Q.-s. Zhang and Zhu 2018)(Gondal et al. 2019), while adversarial robustness has mostly been studied in the realm of classifiers. Few works have systematically evaluated adversarial robustness in generative models, and even less have addressed this question in the context of *multi-view generative models*.

Multi-view generative models, such as the Generative Relational Kernel Machine (Gen-RKM), introduce additional complexity by coupling multiple views (e.g., images and labels) into a single latent representation. This opens new possibilities: consistency across views can in principle act as a natural regularizer, but it may also provide new surfaces for attack. Whether such structure enhances or weakens robustness is an open question that has not been extensively studied before. Furthermore, most existing robustness criteria are tailored to classifiers and single-view generative models, leaving open how to formulate defenses in a multi-view setting.

1.3 Research Question and Objectives

This thesis therefore addresses the following central research question:

Does disentanglement improve robustness in generative representation learning, and how can multi-view consistency be leveraged to strengthen defenses against adversarial attacks?

To answer this, the following objectives were pursued:

- Evaluate the robustness of generative models with varying degrees of disentanglement (VAE, β -VAE, β -TC-VAE) under latent-space adversarial attacks.
- Extend adversarial robustness evaluation to the multi-view Gen-RKM, introducing an adaptation of attack objectives to account for coupled image and label views.

- Propose and assess a novel defense strategy based on enforcing *cross-view consistency*, thereby reducing the success rate of adversarial perturbations.
- Analyze trade-offs in hyperparameter tuning between latent, image-view, and label-view losses, clarifying the practical implications of multi-view coupling.

1.4 Approach

To systematically study these objectives, we designed adversarial attacks directly in the *latent space* of generative models, targeting different reconstruction and consistency losses. Experiments were conducted on the MNIST dataset using both single-view VAE variants and the multi-view Gen-RKM. Robustness was assessed in terms of attack success rate and perturbation cost. Defense strategies were introduced by modifying the Gen-RKM loss function to explicitly enforce cross-view consistency, a novel contribution of this work.

1.5 Contributions

This thesis makes the following key contributions:

1. A systematic evaluation of adversarial robustness in generative models with varying degrees of disentanglement—a setting that has received little attention compared to classifiers.
2. The first empirical study (to our knowledge) of adversarial robustness in the multi-view Gen-RKM, including an extension of attack objectives to account for coupled losses across views.
3. A novel defense mechanism based on cross-view consistency, showing that disentanglement alone does not guarantee robustness, but consistency across views can be exploited as an effective safeguard.

1.6 Structure of the Thesis

The remainder of this thesis is structured as follows:

- Section 2 introduces the key concepts underlying generative modeling, disentanglement, and robustness.
- Chapter 3 reviews related work on generative models, adversarial attacks, and disentanglement.
- Chapter 4 details the methodology, including model selection, attack design, and defense strategies.
- Chapter 5 presents the experimental results and analysis.
- Chapter 6 discusses broader implications, limitations, and avenues for future work.
- Chapter 7 concludes with direct answers to the research question and outlines the main contributions.

2 Background

This section assumes a general background in artificial intelligence and machine learning at the level of a computer science graduate. Its main purpose is to clarify the concepts used in the title and to provide the reader with an intuitive overview of the broader themes that will be discussed later in this thesis.

2.1 Generative Learning

Generative learning is a broad field in AI that focuses on generating new data samples from a probability distribution learned from the original training data. It is distinct from AI techniques such as classifiers and regressors, which aim to predict (or generate, in some sense) a numerical value or a class label, but do not attempt to model or sample from the underlying probability distribution of the training data. The field has risen in popularity in recent years, driven by the widespread use of large language models (LLMs), with ChatGPT being one of the most prominent examples. LLMs that generate text-based responses based on user input are, however, only one example of generative learning. Generative learning can include generating anything from images to audio, videos, or even more general synthetic numerical data.

2.2 Multi-View Learning

Heterogeneous learning, also known as *multi-modal learning*, involves scenarios where multiple sources of information or data modalities are available. Depending on the nature of the task and data, this setting can give rise to different learning problems. Among the most prominent paradigms encountered in such contexts are:

1. **Multi-task learning:** The model is trained to perform multiple related tasks simultaneously. By leveraging shared representations and transferring knowledge across tasks, overall performance can often be improved.
2. **Multi-label learning:** In this setting, each data point may be associated with multiple labels. The model must therefore learn to predict a set of relevant labels rather than a single class.
3. **Multi-view learning:** The model learns from multiple distinct views or representations of the same data point (e.g., different feature sets or modalities). Incorporating multiple views often leads to more robust and informative representations compared to relying on a single source.

This thesis will focus on generative models in a multi-view setting. A basic example of a system situated in this field would be a model that generates 3D shapes based on multiple 2D images taken from different angles. Each image represents a different view of the same object, and by combining these views, the model can learn a more complete and accurate representation for generation.

Multi-view learning has been linked to benefits such as improved generalization and increased efficiency compared to single-view approaches (Chang Xu, Tao, and Chao Xu 2013).

2.3 Disentanglement

Disentanglement is a property of machine learning models that tries to measure how well the model separates different, often human-interpretable, aspects in its representation. A model

being highly disentangled is generally seen as a desirable property and has given rise to different models that try to maximize the disentanglement. One of the reasons that disentanglement is generally seen as such a desirable property is that it makes the model more human-interpretable. If a model is more interpretable it will more easily allow for additional upgrades, tweaks, or refinements on the model. Explainability within AI has been a widely discussed topic in recent years and research into disentanglement is a direct consequence of this. Although disentanglement of machine learning models is a topic with some solid research behind it, there is still no consensus on its definition, how it should be measured, and what a disentangled representation would mean for a model. We discuss some of the different papers on this topic in Section 3.1.

An example of what a disentangled model could look like for the classification of pictures of cats versus dogs is representing it internally with a vector of values directly correlating to the following properties: length of the ears, multiple colours, coloured eyes, tail up, tongue visible. These properties are human-interpretable and will help the model in classifying whether the picture represents a dog or a cat. The model would be considered disentangled if it correctly assigned values to these different characteristics. Whether the model needs to learn what these properties are on its own is again a topic of discussion and will be discussed in section 3.1, but a disentangled representation should be able to assign the correct values to these properties whether these are given in advance or not.

2.4 Robustness in Machine Learning

Robustness in machine learning is a wide and varied topic. We will only show an overview of the subtopics that are interesting in view of the experiments in this paper.

Before looking at the subtopics of robustness, it is essential to understand the importance of evaluating a model’s robustness. Machine learning models, particularly deep learning systems, are now deeply integrated into everyday life — from web searches and customer support interactions to personalized recommendations. As these models become more prevalent, questions arise regarding their reliability and safety. How well do these systems perform under unexpected conditions? And to what extent are they resilient to misuse or adversarial manipulation? For example, an AI-powered customer support agent using inappropriate language when prompted, or a movie recommendation system delivering poor suggestions after someone else has used the account, are both illustrative of undesirable outcomes that can occur when a model lacks robustness under certain expected conditions. Moreover, the widespread deployment of such models increases the risk of malicious exploitation, where systems designed with good intentions may be manipulated by adversarial users to produce unintended or even harmful behavior.

2.4.1 Noise

Noise is a common concept in data-driven fields such as machine learning. It refers to random variations or irregularities in the data that do not reflect the true underlying patterns or signal. Such deviations can arise from measurement errors, data entry mistakes, or inherent randomness in the data-generating or collection process. Noise can obscure meaningful relationships and reduce a model’s ability to generalize.

Robustness to noise is therefore an important property of machine learning models, especially given the imperfect and often messy nature of real-world data. One area where noise plays a critical role is the trade-off between overfitting and underfitting. A model that overfits to noisy data will likely fail to generalize to new, unseen data, while an underfitting model may miss essential patterns entirely. In this way, robustness to noise is often addressed indirectly through regularization techniques and careful model selection aimed at preventing overfitting.

That being said, the noise present in the training data will not always perfectly reflect the noise — and therefore the data — encountered in real-world applications. For example, a change in camera, resolution, or lighting conditions during deployment can significantly affect model performance, especially if such variations were absent during training. Most theoretical frameworks in machine learning acknowledge this by assuming that training and test inputs are drawn from the same distribution. However, in practice, this assumption often fails.

This highlights the importance of designing models that are not only robust to noise in the training data but also capable of generalizing under distribution shifts that arise from natural variations in real-world settings. We will later touch on the role disentanglement may play in this issue in Section 3.1.

2.4.2 Outliers

Outliers can be seen as a more extreme form of noise. They are data points that deviate so strongly from the general trends that they often provide little additional information and may even hinder a model’s ability to learn meaningful patterns. Robustness to outliers therefore refers to a model’s ability to draw reliable and useful conclusions even when some data points deviate significantly from the majority.

A common strategy is to apply outlier detection techniques to filter such points before training. While this can improve model performance, it is not always desirable. For example, when the dataset is small, discarding data may remove valuable information, and in complex domains, outlier detection methods may mistakenly flag valid but rare patterns as outliers.

In these cases, a more effective approach is to design models that reduce the negative impact of outliers while still capturing any potentially useful information they contain. A model that achieves this balance can be considered robust to outliers.

2.4.3 Adversarial Attacks

An adversarial attack on a machine learning model occurs when an attacker deliberately alters input data to fool the model into producing a specific, often incorrect, output. This output can range from a simple misclassification to a response crafted entirely to the attacker’s advantage.

For instance, consider an image classification model trained to distinguish between cats and dogs. By subtly modifying a small set of carefully chosen pixels in a picture of a cat, while keeping the image clearly recognizable to a human observer, the attacker may cause the model to misclassify it as a dog. These perturbations are often imperceptible to humans, which makes adversarial attacks particularly powerful.

As deep learning models are increasingly deployed beyond academic research, robustness against such attacks has become a pressing concern. Adversarial examples are often viewed as worst-case perturbations: while random noise may occasionally induce errors, adversarial methods are designed to maximize the chance of misclassification, thereby exposing a model’s vulnerabilities.

This thesis focuses on adversarial attacks in a generative setting. A more detailed discussion will follow in Section 3.2, together with Figure 3, which illustrates an example of such an attack.

3 Literature review

In this section, we review the most relevant literature related to our work. The discussion is organized around three main themes. First, we examine disentanglement, including its definitions, evaluation methods, and links to interpretability. Second, we discuss robustness, with a

focus on how models handle adversarial attacks and distribution shifts. Finally, we describe the generative models used in this thesis, outlining their key properties and relevance to our research questions. Together, these perspectives provide the theoretical foundation for the experiments and analyses presented in later sections.

3.1 Disentanglement

3.1.1 Definition

Disentanglement is an extensively studied but still often partially redefined subject (Higgins, Amos, et al. 2018)(Carbonneau et al. 2024)(Almudévar et al. 2024). Although exact definitions of disentanglement and how it can be measured have not yet been agreed upon, the general idea stays the same and can be explained intuitively by using representation learning concepts. In the field of representation learning, data can be described as a combination of only factors of variation and nuisances. The factors of variation should resemble conceptually different properties present in the data and should be useful for the task of the model. Nuisances on the other hand should be irrelevant for the task at hand (Higgins, Amos, et al. 2018)(Almudévar et al. 2024). An example could be a dataset composed of images of tops (e.g., t-shirts, sweaters, etc.), where the task of the model is to describe the top in the image. In this case, some of the factors of variation could include the color, size, presence of sleeves, and the material of the top. Nuisances, on the other hand, could include elements such as the background or the facial expression of the person wearing the garment.

The intuitive definition mentioned above and the slight variations thereof, where nuisances are not explicitly mentioned, are widely used in the older Desjardins, Courville, and Bengio (2012) and more recent Almudévar et al. (2024) literature surrounding the topic. One of the reasons why it is difficult to give a more specific definition is that defining the world structure in a formal way is a hard task, especially given that it must be widely applicable to any task or model (Higgins, Amos, et al. 2018).

For that reason, this intuitive definition is the one that is generally used to understand and describe disentanglement and if we were to accept a standardized definition of disentanglement, this definition would likely form a strong basis, as it already does in the attempts that have been made. Those attempts were generally crafted in tight cohesion with a quantitative model to measure disentanglement, sadly most measurements miss something that seemed essential to other people engaging with the subject, which leads to a different definition (Carbonneau et al. 2024)(Almudévar et al. 2024).

The goal of giving a more precise definition was to explain what it concretely means for factors of variation to be separated and how we can measure if they actually are separated. Before these attempts were made and used the main way people looked at disentanglement was through visual inspection, where it was first discovered that having a disentangled representation might have some serious benefits including the human interpretability. Some of the benefits that have been reported were an increased predictive performance and reduced sample complexity among others (Carbonneau et al. 2024). These benefits show that having a specific way to measure and later optimize for disentanglement can be rewarding.

The properties that represent a disentangled representation are made more explicit in Carbonneau et al. (2024). The properties mentioned in this paper are generally used or adapted in other papers to give the exacter definition and introduce metrics. In general the properties allow for a common vocabulary in the field making it easier to interpret and compare. The paper lists the following two bigger properties of disentanglement, firstly the representation should be distributed and secondly it should have information that is relevant for the task at hand, these

properties will be referred to as *Factor Independence in Representation* and *Information Content* respectively.

Factor independence looks at the different factors and makes sure that one does not affect the other. The factors of variation should also be independent in the representation space, indicating that if one factor changes, not only should other factors not be affected, the representation space should also only be affected in terms of this factor and no other factors. This property is commonly referred to as modularity in Ridgeway and Mozer (2018), or disentanglement in Eastwood and Williams (2018). Another commonly introduced part of factor independence, which is referred to as compactness in Ridgeway and Mozer (2018) and completeness in Eastwood and Williams (2018), points to the amount of representation space that is needed to describe a single factor. It is mentioned that ideally this should take only one dimension (Carbonneau et al. 2024).

Information content is the term used to describe what information is actually stored in the representation code. It is also referred to as explicitness in Ridgeway and Mozer (2018). This property, when correctly measured, would allow us to check if a single representation actually includes all the necessary explanatory factors that are of importance to the task at hand. Without this property, one could perfectly encode a synthetic model that achieves perfect factor independence, but does not encode any useful information.

These concepts provide the foundation for understanding disentanglement and naturally lead into the discussion of how disentanglement can be quantitatively measured.

3.1.2 Measuring Disentanglement

Over the years, different frameworks have been proposed to measure disentanglement. Several reviews have analyzed the metrics used, attempting to classify them and clarify what aspects of disentanglement they capture and how comprehensively they do so. In this work, we focus on the framework implemented by Eastwood and Williams (2018), which is widely adopted in the literature due to its simplicity and the availability of supporting code.

The framework measures both *Factor Independence* and *Information Content*, dividing them into three categories. Factor independence is further split into *disentanglement* and *completeness*, while information content is captured by *informativeness*. Disentanglement and completeness represent two sides of the same coin: we aim for one latent variable to correspond to a single generative factor (disentanglement) and for each generative factor to be represented by only one latent variable (completeness). Disentanglement therefore describes how well each latent dimension encodes a single generative factor, while completeness measures how uniquely each factor is encoded in the latent space.

All of these properties are measured using regressors. Both Lasso and Random Forest regressors are employed to provide different levels of complexity in the regression models. The detailed theoretical and mathematical background of the framework is omitted here, as it does not contribute to understanding how the framework was applied in this study.

3.2 Adversarial robustness

In this thesis, we focus primarily on robustness in the context of adversarial attacks. The attacks considered are largely based on Kos, Fischer, and Song (2018), which provides a comprehensive overview of adversarial techniques for generative systems. Understanding the methods described in this work is crucial for interpreting the extensions introduced in Section 4.5.

It is important to note that most adversarial studies to date have focused on discriminative models, such as classifiers. This distinction is critical because generative systems operate on

fundamentally different principles, often relying on probability distributions rather than direct label prediction. Consequently, much of the literature on adversarial robustness does not directly apply to the generative models studied in this thesis. From this point onward, unless stated otherwise, all references to adversarial robustness will specifically concern generative systems.

In Carlini, Athalye, et al. (2019), they give a detailed overview on how adversarial robustness should be measured and what some of the pitfalls are. Section 4.5 will take these into account to avoid the most common mistakes made in academic papers on the topic of adversarial robustness. Although Carlini, Athalye, et al. (2019) focuses most on researchers that claim to have built fully or partially adversarially robust models, it is also mentioned they are not the only ones that can benefit from taking into account the different notions that were highlighted. It addresses two bigger topics; first, it provides guidelines on how one should perform defense evaluations. Secondly, it addresses common pitfalls for defense evaluations. As this thesis is not aiming to provide a defense against adversarial attacks but rather to evaluate adversarial robustness in different models, these topics will not provide a guideline as they would for people implementing defenses, but they will still cover topics that touch on evaluating adversarial robustness as a whole.

3.2.1 Threat models

One of the commonly mentioned topics that Carlini, Athalye, et al. (2019) touches on is threat models. A threat model should specify the assumptions that are made about the adversary, these assumptions could include their goals and knowledge about the system. They claim that a threat model is necessary for a multitude of reasons, an important reason being that any scientific research should be falsifiable. Although this paper will not try to claim complete robustness, but only comparative robustness, the claims made should still be falsifiable, and for that to be possible, the claims that are made must be specific enough to make it possible to build an experiment contradicting the claims, given the correct results. Another reason why threat models might be interesting is the insight that they bring into adversarial attacks on a system in the correct setting. Concerning the threat model, Kos, Fischer, and Song (2018) also gives a motivating attacking scenario in the case of generative models, which is the domain confronted in this thesis. Such an attacking scenario is not only crucial to understanding why an attacker would be interested in such a system, it also shows that these attacks have real world examples. The following scenario adjusted from the example in Kos, Fischer, and Song (2018) gives a general overview of how one might attack a generative system.

3.2.2 Attacking scenario

Consider the following three parties in this scenario, there is a sender, a receiver and the attacker. The sender and receiver want to share images using a computer network, they would like to save on bandwidth of the system by only sending a representation of the picture, they do this by only sending a latent vector z , that represents the picture. This latent vector can be the representation of the picture by any model trained on the input distribution containing the images, but Kos, Fischer, and Song (2018) mentions a VAE, so we will do the same. This VAE model is shared between them, so in a normal situation the sender can encode the picture using the VAE and the receiver would regenerate the image from the latent representation using the same model. The goal of the attacker in this scenario is to make the sender send a picture that the attacker chose to the receiver, with the sender thinking that they sent a picture of their own liking. It should be mentioned that the attacker is not trying to trick the sender psychologically, nor is he directly in control over the bytes sent to the receiver. The attacker instead is presenting the sender with an image x^* , this x^* looks almost identical to the image x , which is the image that

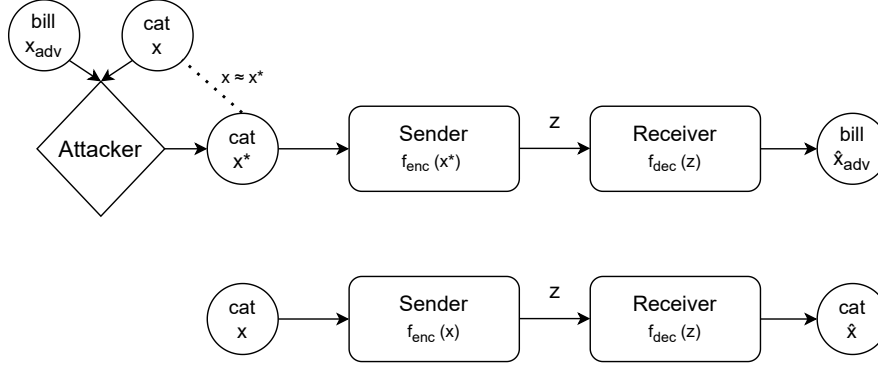


Figure 1: An attacking scenario based on Figure 1 in Kos, Fischer, and Song (2018). The representation learning model is common knowledge among the sender, receiver and attacker. The above diagram shows a successful attack and the bottom diagram shows how it works without an attacker in action. In an attacking scenario, the attacker provides the sender with a picture similar to the picture that they would like to send, but adversarially perturbed. The sender sends the encoding z over the network to the receiver. The receiver reconstructs the image using the model, but gets to see a picture of the attackers liking instead of the intended picture.

the sender would like to share with the receiver. The sender unknowingly encodes x^* instead of x and sends the latent representation z through to the receiver, when the receiver regenerates the image, it shows a completely different image from the one that was intended.

A concrete example shown in figure 1 based on an illustration introduced in Kos, Fischer, and Song (2018) displays a scenario, both with and without an attacker interrupting the process. The process seen in the Figure 1 can be described as follows: The sender wants to send an image of a cat to the receiver, we call this image x . The attacker presents a picture of a cat to the sender, the sender chooses to share this particular one with the receiver, not seeing or knowing that it is x^* and not the desired x . The sender sends the latent representation z to the receiver. The receiver reconstructs the image from the latent representation z , but instead of getting a reconstruction of the cat picture, the receiver gets a different picture chosen by the attacker, which could f.e. be a request to send cryptocurrency to the attacker.

This is not the only example where similar attacks can happen, as long as a VAE (or a different representation model) is used for its latent representation in storage or transfer of the data in any way, the shared data may be attacked without the knowledge of the sender (Kos, Fischer, and Song 2018). It is also mentioned that the type of compression algorithms used in this scenario have already been proposed in works such as Toderici et al. (2017), indicating that research on the topic could have valuable impact on future work.

3.2.3 Threat model assumptions

To continue building on the threat model, more assumptions taken from Kos, Fischer, and Song (2018) have to be made explicit as suggested in Carlini, Athalye, et al. (2019). We follow the standards used in Kos, Fischer, and Song (2018) that state that the target generative model of the adversarial attack is represented by $G_{targ}(x) = f_{dec}(f_{enc}(x))$. We assume that G_{targ} is available to the attacker like we did in the example depicted in Figure 1. We also have that the model is trained on data that can be labeled by humans into meaningful and different classes, these labels do not explicitly have to be part of the data. The model G_{targ} should be a good

generative model in the sense that it can generate an output of the class that was given as the input, for example $\hat{x} = G_{\text{targ}}(x)$, should be of class y when x is of this same class. It is also stated that this target output class does not have to be from the most salient class, indicating that for the MNIST dataset a sample could be of a class defined by the style of the digit rather than the digit itself, which represents the most salient class. Lastly the attacker must have access to some samples of the same distribution used in training, where the samples have the possible target labels that the attacker intends to generate. These samples must not be labeled, similarly to the training of the model.

3.2.4 Generating the adversarial example

The way an attacker attacks the representation learning model is by generating adversarial examples for it, these can be acquired in a multitude of different ways. The fact that there is no single uniform way to generate adversarial examples makes the problem of finding appropriate defenses to them, that much harder and has led to works such as Carlini, Athalye, et al. (2019). One of the attacks that is often implemented involves solving an optimization problem that aims to change the original source image x to x^* , where the difference between the two is kept to a minimum and the reconstruction of $G_{\text{targ}}(x^*)$ is considered to be of a different class in an untargeted attack and of the target class in a targeted attack. In Kos, Fischer, and Song (2018) they mention an Oracle, which is a term used to describe the decider, which can tell the difference between the target class and other classes. This Oracle can be the attacker if only a single successful attack is needed, but it is often a classifier trained by the attacker. This classifier can be used to generate multiple adversarial examples and it can streamline the process of attacking the model and evaluating the attack.

We will go over the three different attacks mentioned in Kos, Fischer, and Song (2018) and expand on the last one as it matches the attacks implemented in this paper.

The classifier attack is the first one mentioned and although it is the worst performing (Kos, Fischer, and Song 2018), it lays a relatively intuitive foundation. The problem of finding adversarial examples to classifiers has been studied in many different papers such as Goodfellow, Shlens, and Szegedy (2015) and Moosavi-Dezfooli, Fawzi, and Frossard (2016) with some significant improvements over the years (Abdollahpoorrostam, Abroshan, and Moosavi-Dezfooli 2024) and it has been shown that there exist a variety of attacks that work well on them. The classifier attack tries to reduce the problem of finding adversarial examples to generative models to the problem of finding adversarial examples to deep neural networks that focus on classifying to try and capitalize on the advancements in this field. For this attack the attacker trains a classifier for the classes he wants to target and therefor discriminate against. The classifier should take the latent representation z as input and output the class it belongs to, target class or not. The training data for this classifier should be approximately the same as the input data with the additional labels for the target class and its complement. This classifier can afterwards be used in combination with the optimization problem described in 3.2.5 to get adversarial examples. Note that Kos, Fischer, and Song (2018) mentioned that this attack does not perform very well, partly due to the classifier wrongly classifying for a great number of cases.

The next attack mentioned is the $\mathcal{L}_{\text{VAE}}$ attack. This approach is named after the loss function used in the VAE model used throughout the paper, because it uses its loss function in the adversarial generation process. In this approach the attack calculates the full reconstruction beforehand and optimizes for it. Concretely, the VAE loss function is used to compute the reconstruction loss between the reconstruction of the adversarial example and the reconstruction of the target. This requires calculating these reconstructions in every step of the optimization process, making it the slowest of the methods mentioned in Kos, Fischer, and Song (2018).

The most effective attack and the most important one for this paper is the latent attack mentioned in Kos, Fischer, and Song (2018). It describes an attack that targets the latent space of the generative model by computing the loss function $\mathcal{L}_{\text{latent}} = L(z_t, f_{\text{enc}}(x^*))$ where z_t is the encoding of x_t and x^* is the adversarial example. $L(\cdot, \cdot)$ describes the distance measure and can be initialized to the L_2 norm, following the common assumption of an approximately euclidean space. Starting from a source image and using the techniques mentioned in 3.2.5, we optimize the adversarial example until the necessary conditions are met.

3.2.5 Solving the Optimization Problem

Different methods can be used to solve the optimization problem, but we will focus on the optimization method L_2 , as it is a known metric and performs consistently better than the fast gradient sign (Goodfellow, Shlens, and Szegedy 2015)(Kos, Fischer, and Song 2018). The optimization problem can be described as:

$$\operatorname{argmin}_{x^*} \lambda L(x, x^*) + \mathcal{L}(x^*, y_t) \quad (1)$$

as explored in Szegedy et al. (2014), Carlini and Wagner (2017) and taken directly from Kos, Fischer, and Song (2018). Here, $L(\cdot, \cdot)$ describes the distance measure, and $\mathcal{L}$ describes the loss function used, pointing to one of the aforementioned methods such as $\mathcal{L}_{\text{latent}}$. Lastly, λ is the constant that allows balancing between, for one, the amount of change between the original and adversarial example, and, for the other, the loss in reconstruction. The optimization problem itself can be solved with a multitude of different solvers, among which Adam is one of the most prominent (D. Kingma and Ba 2014).

3.3 The Machine Learning models

Before presenting the individual models, we note that the selection of baseline autoencoders follows the experimental setup of the original Gen-RKM paper (Pandey, Schreurs, and Suykens 2021). Specifically, the VAE, β -VAE, and β -TCVAE were chosen in that work to evaluate disentanglement performance across standard generative models. By including the same baselines in our study, we ensure that our robustness experiments are comparable with the disentanglement results reported in prior work, and that any differences in performance can be attributed to our focus on adversarial robustness rather than differences in model selection.

3.3.1 Variational Autoencoders (VAEs)

One of the most academically important generative models is the variational autoencoder, originally introduced by D. P. Kingma and Welling (2013). VAEs combine ideas from probabilistic modeling and deep learning to learn how to generate new data that resembles the input data. They do this by encoding the input into a compressed, latent representation and then decoding this representation back to reconstruct the original data.

Unlike traditional autoencoders, VAEs learn a probabilistic mapping between the input and the latent space. This allows them to model uncertainty and generate new samples by sampling from the learned latent distribution. The training process optimizes both how well the model can reconstruct the input and how closely the learned latent distribution matches a predefined prior, typically a simple Gaussian.

This balance ensures that the latent space is continuous and structured, making it possible to generate meaningful new data points by sampling or interpolating between points in this space. Because of these properties, VAEs have been widely used for tasks such as image generation, anomaly detection, and representation learning.

This model has been extensively used as a reference point to evaluate and analyze various aspects of generative models in multiple subfields, including disentanglement (Eastwood and Williams 2018), optimization techniques (D. Kingma and Ba 2014), and adversarial examples (Kos, Fischer, and Song 2018). Consequently, it is a well-established framework for conducting experiments, which is precisely the approach adopted in this thesis.

3.3.2 β -VAE

β -VAE is a variant of the variational autoencoder introduced by Higgins, Matthey, et al. (2017). It modifies the original VAE framework by placing greater emphasis on the regularization term in the training objective, controlled by a parameter β . By increasing this weight, the model encourages the latent space to capture more disentangled and interpretable factors of variation in the data. This adjustment often comes at the cost of reconstruction accuracy but can lead to more meaningful and structured latent representations, making β -VAE particularly useful for representation learning and tasks requiring interpretability.

3.3.3 β -TCVAE

Several extensions of the standard VAE and β -VAE have been proposed, each targeting specific limitations or seeking improvements in certain aspects of the model. One such variant is β -TCVAE, introduced by Chen et al. (2018). This approach builds upon the disentanglement focus of β -VAE but explicitly penalizes the total correlation in the latent variables. By reducing statistical dependencies between latent dimensions, β -TCVAE encourages more factorized and independent representations, often leading to improved disentanglement without excessively compromising reconstruction quality.

3.3.4 Generative Restricted Kernel Machines (Gen-RKM)

The Generative Restricted Kernel Machine (Gen-RKM), introduced by Pandey, Schreurs, and Suykens (2021), is a framework that extends Restricted Kernel Machines (RKM) (Suykens 2017) to a generative setting. It enables multi-view generative learning by using a shared latent representation across different views of the data. This allows the model to generate one view given another, similar to the cross-view tasks often studied in multi-modal learning.

At the core of Gen-RKM is the eigen-decomposition of the kernel matrix, which defines the latent variables. These eigenvectors are mutually orthogonal, enforcing a form of uncorrelated feature learning that can be interpreted as a disentanglement mechanism. This aligns well with the goals of our thesis, which investigates the effect of disentanglement on robustness.

One of the strengths of the framework is its dual formulation. It can be implemented using classical kernel methods or deep neural networks as explicit feature maps, which allows it to scale to more complex data, such as images. When using deep feature maps, Gen-RKM jointly learns the representation and the shared latent space through a tailored training procedure that respects the kernel-theoretic foundation.

In this thesis, we adopt Gen-RKM as the backbone generative model due to its ability to:

- handle **multiple views** of the data explicitly
- introduce **disentanglement** via orthogonal latent factors
- allow flexibility in architecture choice (kernel or deep).

An integral aspect of the Gen-RKM framework that will be of importance in the adversarial attacks is the interplay between the common subspace, the feature maps, and the pre-image maps.

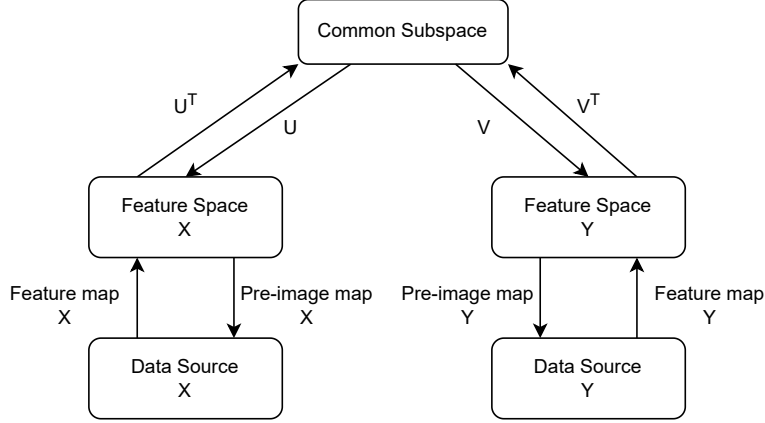


Figure 2: Simplified overview of the Gen-RKM architecture. Adapted from Figure 1 in Pandey, Schreurs, and Suykens (2021).

Figure 2 shows the interplay between the different conceptual aspects of Gen-RKM in a simplified version. Most notably are the learned interconnection matrices U and V . These matrices allow for a translation from the latent variables to the different feature spaces and the other way around. How these different aspects are learned is not essential to understanding this thesis, but one should know that the interconnection matrices and pre-image maps are learned, but the feature map itself is chosen by picking an appropriate kernel function based on the task of the model.

This model is also chosen because it has a mathematical disentanglement background that has been measured in the paper. We can use the results produced by them, namely that the Gen-RKM framework is in many cases and under the framework of Higgins, Amos, et al. (2018) often more disentangled than the other models (VAE, B-VAE, B-TCVAE and info-GAN), especially when the latent space dimension is chosen to be close to the amount of disentangled generating factors.

3.3.5 LeNet

The LeNet architecture, introduced by Lecun et al. (1998), was one of the first successful convolutional neural networks (CNNs) and a landmark in the application of gradient-based learning to visual pattern recognition. Initially developed for handwritten digit recognition on the MNIST dataset, LeNet demonstrated the effectiveness of end-to-end learning in extracting hierarchical spatial features directly from raw pixel data, without requiring handcrafted feature extraction.

The original LeNet-5 consists of two convolutional layers, each followed by a subsampling (average pooling) layer, and two fully connected layers leading to a final softmax classification layer. Convolutional layers learn localized edge and texture detectors, pooling layers provide translation invariance and reduce computational complexity, and the fully connected layers integrate the learned features for classification. At the time of its introduction, this architecture was

computationally feasible due to its relatively small number of parameters, making it practical for the hardware available in the late 1990s.

Despite the development of deeper and more complex CNN architectures, LeNet remains a widely used benchmark model, particularly for grayscale, low-resolution image datasets such as MNIST. Its simplicity and efficiency make it an appealing choice for experiments requiring a stable and well-understood baseline. In this work, LeNet serves as a reference classifier to evaluate downstream performance and robustness in the presence of adversarial perturbations and representation disentanglement constraints.

4 Methodology

4.1 Overview

The goal of this thesis is to determine whether the degree of disentanglement of a generative model has a positive influence on robustness against adversarial attacks in a multi-view setting.

To investigate this, we develop adversarial attacks on different generative models, measure their robustness, and compare the results across models. The level of disentanglement provides insight into the research question and allows us to assess the relationship between robustness and disentanglement.

For the robustness evaluation, the VAE, β -VAE, and β -TCVAE are included as baseline autoencoders. These models were selected in line with the original Gen-RKM study (Pandey, Schreurs, and Suykens 2021), which reported their disentanglement performance. We train them only once on the MNIST dataset, leveraging their established disentanglement characteristics, so that all subsequent experiments can focus solely on robustness while keeping the baseline model properties fixed.

4.2 Datasets

4.2.1 MNIST

For the experiments, we use the MNIST dataset (Lecun et al. 1998), a standard benchmark in machine learning research. MNIST consists of 70,000 grayscale images of handwritten digits (0–9), each with a resolution of 28×28 pixels. The dataset is divided into 60,000 training samples and 10,000 test samples.

No additional preprocessing was applied; the models are trained directly on the raw pixel intensities in the range $[0, 255]$. The dataset was chosen due to its accessibility, widespread use in generative modeling and disentanglement studies, and its suitability for evaluating robustness, since its simple structure makes perturbations and reconstruction errors easy to interpret.

4.3 Model Architectures

The models that were used in the experiments have been briefly described in section 3. The actual implementation of them and the necessary parameters will be described in this section for transparency and replicability reasons.

4.3.1 The Autoencoders (VAE, β -VAE, β -TCVAE)

The autoencoders were adapted from the open-source implementations of Chadebec, Vincent, and Allasonniere (2022), using the basic reference notebooks provided in the official GitHub

repository (Chadebec 2022). These implementations follow standard formulations of the Variational Autoencoder (VAE), the β -VAE, and the β -TCVAE, making them directly applicable for disentanglement and robustness evaluation in our framework.

For each of the autoencoder models (VAE, β -VAE, and β -TCVAE), we used the official implementations from Chadebec, Vincent, and Allasonniere (2022) via their public GitHub repository (Chadebec 2022). The models were trained on the MNIST dataset (Lecun et al. 1998) for 20 epochs. This number of epochs provided a good balance between training time and reconstruction quality, as both training and validation losses showed signs of convergence. Longer training did not yield substantial improvements for any of the autoencoders according to the validation loss as shown in Section B.

Hyperparameters. The β -VAE was trained with a disentanglement weight of $\beta = 2$, applied uniformly to the Kullback–Leibler (KL) divergence term. For the β -TCVAE, the KL divergence was decomposed into mutual information (I), total correlation (TC), and dimension-wise KL terms, with weights $(\alpha, \beta, \gamma) = (1, 2, 1)$. These settings follow common practice in disentanglement literature (Chen et al. 2018), and the selection of baseline models and hyperparameters is consistent with the original Gen-RKM paper (Pandey, Schreurs, and Suykens 2021), with the exception that in our experiments the total correlation weight β was slightly reduced (2 instead of 3) while α and γ match the original paper. This minor difference is not expected to significantly affect the models’ disentanglement or their role as baselines for robustness evaluation. All hyperparameters were kept fixed across experiments, as these autoencoders are included solely as baselines for comparison.

4.3.2 Generative Restricted Kernel Machines (Gen-RKM)

The Generative Restricted Kernel Machine (Gen-RKM) framework, introduced by Pandey, Schreurs, and Suykens (2021), combines principles from kernel methods and neural networks to model data in a generative setting. The authors provide code and pretrained models to support reproducibility and further academic study. Within our framework, Gen-RKM is employed alongside autoencoders to enable a comparative evaluation of robustness and disentanglement.

For the Gen-RKM model (Pandey, Schreurs, and Suykens 2021), we used the publicly available pretrained models provided by the authors, downloaded from (Pandey, Schreurs, and Suykens 2021b). Two pretrained models were employed: one trained on the MNIST dataset (Lecun et al. 1998) and one on the CelebA dataset (Liu et al. 2015). Both models were used as-is, without additional retraining or modification of hyperparameters, to maintain consistency with the original published results.

4.3.3 LeNet Classifier

Architecture For baseline classification, we employ the LeNet architecture (Lecun et al. 1998), originally designed for handwritten digit recognition on MNIST. The network consists of two convolutional layers with ReLU activations, each followed by 2×2 max pooling, and two fully connected layers leading to a softmax output over the ten MNIST classes. The first fully connected layer has 128 neurons. A detailed overview is provided in Table 1.

Training Setup The network is trained end-to-end using cross-entropy loss and the Adam optimizer with a learning rate of 0.001, using a batch size of 64. Training runs for a maximum of 50 epochs, with early stopping applied if validation accuracy does not improve for three consecutive epochs. The concrete training metrics can be found in Appendix A where Figure 5

Table 1: LeNet Architecture for MNIST Classification

Layer	Details	Output Shape
Conv1	6 filters, 5×5 kernel, ReLU	$28 \times 28 \times 6$
MaxPool1	2×2 , stride 2	$14 \times 14 \times 6$
Conv2	16 filters, 5×5 kernel, ReLU	$10 \times 10 \times 16$
MaxPool2	2×2 , stride 2	$5 \times 5 \times 16$
FC1	Fully connected, 128 neurons, ReLU	128
FC2	Fully connected, 10 neurons, Softmax	10

shows the validation accuracy during training and table 7 shows how it performed on the test set.

Experimental Role This architecture provides a stable baseline for evaluating the impact of more complex interventions, such as disentanglement regularization or adversarial perturbations, on downstream tasks.

4.4 Disentanglement and Latent Space

4.4.1 Measuring Disentanglement

In Pandey, Schreurs, and Suykens (2021), disentanglement was measured using the framework proposed by Eastwood and Williams (2018). We take the reported values directly from this work as the basis for our assessment of disentanglement. Following their results, the GEN-RKM framework is generally more disentangled than the other models studied when the latent dimensionality is close to the number of generating factors. The implementation of Gen-RKM was compared with the VAE, β -VAE, and β -TCVAE models in terms of disentanglement in Pandey, Schreurs, and Suykens (2021), and these results are used to contextualize our robustness evaluation. Although more recent disentanglement frameworks, such as Almod var et al. (2024), may provide metrics that better align with the current understanding of disentanglement, applying them to the full set of models considered in this work falls outside the scope of this thesis.

4.4.2 Latent Dimensionality and Generating Factors

For the MNIST dataset, the number of generative factors is approximated to be 10 or less, representing both the digit identity and major style variations across digits (Cheung et al. 2015). The pretrained Gen-RKM model internally computes kernel PCA components resulting in intermediate representations of higher dimension (e.g., $h \in \mathbb{R}^{5000 \times 500}$). However, the actual latent dimensionality used for the compressed representation is determined by the model hyperparameter $h_dim = 10$, which sufficiently aligns with the number of generating factors. This correspondence ensures that the latent space is appropriately sized to capture the key generative factors and justifies the use of the pretrained model and its reported disentanglement characteristics as a valid baseline for our robustness evaluation.

4.5 Latent Attack Framework

4.5.1 General Attack Definition & Pseudocode

For measuring robustness, we implemented the latent attack as described in Kos, Fischer, and Song (2018), with slight modifications to emphasize the multi-view aspect of our model. Among

the different adversarial methods discussed in the paper, the latent attack consistently showed superior performance and was characterized as the most effective across models and datasets. This motivated our choice to focus exclusively on this method, rather than the classifier or $\mathcal{L}_{VAE}$ attacks also mentioned.

Following Kos, Fischer, and Song (2018), we consider adversarial perturbations that operate in the latent space of a generative model rather than directly on the input. These *latent attacks* aim to construct adversarial examples $\mathbf{x}^*$ that remain visually similar to a source image $\mathbf{x}_s$, while their latent encoding is forced to align with that of a chosen target image $\mathbf{x}_t$. This makes them particularly well-suited for probing the stability of the shared latent space in our multi-view generative setting.

Concretely, we first compute the latent representation of the target image as

$$\mathbf{z}_t = f_{\text{enc}}(\mathbf{x}_t).$$

The adversarial example is initialized as $\mathbf{x}^* = \mathbf{x}_s$, and then iteratively updated to minimize the discrepancy between its encoding and the target code:

$$\mathcal{L}_{\text{latent}} = L(f_{\text{enc}}(\mathbf{x}^*), \mathbf{z}_t),$$

where $L(\cdot, \cdot)$ denotes a generic distance measure in latent space.

Note that this implementation differs from the original formulation by Kos, Fischer, and Song (2018) in one crucial aspect, which is λ . For all of our experiments, λ is set to 0. This change is balanced out by the attack success criteria and early stopping mentioned in Section 4.5.4. The stopping criterion allows the attack to carry out only for as long as is strictly necessary and therefore only changes as much as is needed even without the use of λ .

Algorithm 1 gives an overview of the most basic latent space attack, leaving out extensions and specific model representations. It should be noted that throughout the paper, we assume the attacker has full white-box access to the model (cf. Section 3.2.3), this is also indicated by $\mathcal{M}$ being an input requirement in algorithm 1.

Algorithm 1 General Latent Attack

Require: Source input x_s , target input x_t , model $\mathcal{M}$, steps T , learning rate α

Ensure: Adversarial example x^*

```

1:  $z_t \leftarrow \mathcal{M}.\text{get\_latent}(x_t)$  ▷ latent representation of target
2:  $x^* \leftarrow x_s$  ▷ initialize adversarial example
3: Enable gradient tracking on  $x^*$ 
4: Initialize optimizer with learning rate  $\alpha$ 
5: for  $t = 1$  to  $T$  do
6:   optimizer.zero_grad()
7:    $z^* \leftarrow \mathcal{M}.\text{get\_latent}(x^*)$  ▷ latent of current adversarial example
8:   loss  $\leftarrow L(z^*, z_t)$  ▷ generic latent-space distance
9:   Compute gradients and update  $x^*$  via optimizer
10: end for
11: return  $x^*$ 
```

After creating the adversarial example (x^*) through optimization, $\mathbf{x}^*$ passes through the encoder-decoder pipeline. This produces an *adversarial reconstruction*, i.e., the decoded version of the perturbed encoding. In a successful targeted attack, this reconstruction should resemble the target class rather than the original source class.

To quantify the success of an attack, we follow Kos, Fischer, and Song (2018) and employ an auxiliary classifier f_{class} . The classifier is applied both to the adversarial reconstruction and to the adversarial example itself, leading to two complementary criteria for a successful attack:

1. **Target alignment:** the adversarial reconstruction should be classified as belonging to the target class,

$$f_{\text{class}}(f_{\text{dec}}(f_{\text{enc}}(\mathbf{x}^*))) = y_t,$$

where y_t denotes the target label.

2. **Source consistency:** the adversarial example should remain visually close to the source and still be classified as such,

$$f_{\text{class}}(\mathbf{x}^*) = y_s,$$

where y_s is the label of the source image.

This two-part definition ensures that an adversarial attack both (i) succeeds in steering the reconstruction towards the target class, and (ii) preserves the perceptual similarity of the perturbed input to its original source.

Figure 3 shows the adversarial example and its reconstruction for a latent attack on an MNIST datapoint, serving as a visual aid to understand the attack process.

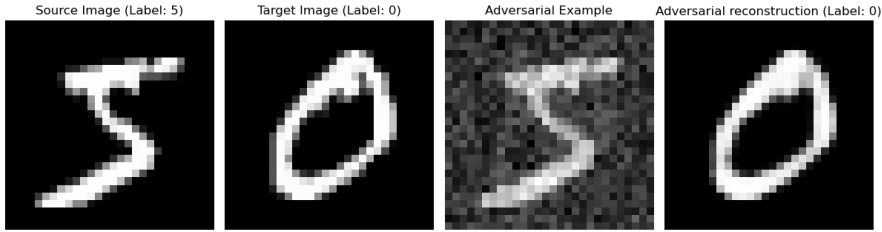


Figure 3: Illustration of a latent attack on MNIST, with 5 as the source image and 0 as the target image. The adversarial reconstruction resembles the target class (0) rather than the original source (5).

4.5.2 Adaptation for Autoencoders

For our experiments with autoencoders, we adapted the general latent attack framework described in Section 4.5.1. The core structure of the attack remains unchanged: we iteratively update a source input $\mathbf{x}_s$ to minimize the discrepancy between its latent representation and that of a target input $\mathbf{x}_t$. This ensures that the adversarial example steers the reconstruction towards the target while remaining perceptually similar to the source.

The main adaptation for autoencoders is the use of the mean vector μ from the encoder output as the latent representation. In variational autoencoders, the encoder outputs a distribution (typically a mean and variance), but for deterministic reconstruction in the adversarial setting, we optimize with respect to the mean μ directly. In addition, we removed references to latent-space samplers, focusing purely on the deterministic case.

Formally, given a trained autoencoder with encoder f_{enc} and decoder f_{dec} , the adversarial example is initialized as $\mathbf{x}^* = \mathbf{x}_s$ and iteratively updated to minimize

$$\mathcal{L}_{\text{latent}} = L\left(f_{\text{enc}}(\mathbf{x}^*)_{\text{embedding}}, f_{\text{enc}}(\mathbf{x}_t)_{\text{embedding}}\right),$$

where $L(\cdot, \cdot)$ denotes a generic distance measure in latent space.

The optimization procedure, including gradient computation and update steps, follows Algorithm 1, with the only difference being that the latent vectors correspond to the encoder’s mean output rather than samples from a latent distribution. All other aspects, such as initialization, loss computation, and iteration over steps, remain consistent with the general framework.

This adaptation ensures that the latent attack is directly compatible with autoencoders while maintaining the targeted adversarial behavior described previously.

4.5.3 Adaptation for Gen-RKM

The latent attack framework can be directly adapted to the Generative Relational Kernel Machine (Gen-RKM) setting with minimal modification. As in the autoencoder case, the goal remains to generate an adversarial example $\mathbf{x}^*$ from a source input $\mathbf{x}_s$ such that its latent representation aligns with a target $\mathbf{x}_t$, while $\mathbf{x}^*$ itself remains perceptually similar to $\mathbf{x}_s$.

In Gen-RKM, the latent space is defined via a feature map $\phi(\cdot)$ and a learned projection matrix U , which together yield the latent representation for the target image as defined in line 1 in algorithm 1.

$$\mathbf{h}_t = U^\top \phi(\mathbf{x}_t),$$

where $\phi(\cdot)$ is the output of the learned neural network **net1** and U is the learned latent projection matrix.

The adversarial example is initialized as $\mathbf{x}^* = \mathbf{x}_s$ and iteratively updated to minimize the discrepancy in the latent space:

$$\mathcal{L}_{\text{latent}} = L(\mathbf{h}^*, \mathbf{h}_t), \quad \mathbf{h}^* = U^\top \phi(\mathbf{x}^*),$$

The algorithm preserves the core principles of the latent attack: the adversarial perturbation operates in the latent space, and the source input remains visually consistent with its original class. Algorithmically, the attack follows the same structure as in the autoencoder case, replacing the encoder with the Gen-RKM’s feature extractor and linear projection.

This adaptation ensures that latent attacks can probe the stability and robustness of Gen-RKM’s learned latent space in a comparable manner to traditional autoencoders, making it a natural extension of the latent attack framework described in Section 4.5.1.

4.5.4 Attack Extensions

We propose two extensions to the basic latent-space attack framework to improve adaptivity and leverage multi-view representations.

Classifier-based stopping criterion (general). Instead of running the attack for a fixed number of iterations, we stop the optimization once an external classifier f_{class} predicts the target label y_t on the *reconstruction* of the perturbed input, $\hat{x}^*$. Formally, the attack proceeds until

$$f_{\text{class}}(\hat{x}^*) = y_t \quad \text{or a maximum number of steps is reached.}$$

While this increases computational cost due to the need to compute the adversarial reconstruction at each step, it ensures that the attack adapts to the model’s decision boundary and terminates as soon as the target class is achieved, independent of the optimizer or a fixed number of iterations.

This extension can be applied to any latent-space attack, whether for standard autoencoders or Gen-RKM. It allows the attack to adapt to the model’s decision boundary and terminate as soon as the target class is achieved, potentially reducing unnecessary iterations. Moreover, it

enables a more consistent evaluation of the attack, since stopping is determined by the classifier rather than by the optimizer choice or a fixed number of steps. Note that computational speed is not a concern for this thesis; the extension is primarily intended to be used in combination with metrics, providing less arbitrary and more meaningful evaluation thresholds.

Multi-view latent targeting (Gen-RKM specific). The latent attack framework for Gen-RKM already operates across multiple views via the shared latent representation h^* . This extension makes it possible to explicitly target individual views in addition to the common latent space. For a given input x_s , the adversarial example is initialized as $h^* = U^\top \phi(x_s)$, where U and ϕ define the latent space as usual.

For each view v , let $V^{(v)}$ be the learned projection from the common latent space to the feature space of view v . The feature-space representation of the adversarial example in view v is then

$$\tilde{\phi}^{(v)}(x^*) = V^{(v)}h^*.$$

Note that $\tilde{\phi}^{(v)}(x^*)$ should not be confused with the standard feature mapping $\phi^{(v)}$; rather, it represents the feature mapping of the input in one view into the feature space of a different view, obtained via the corresponding projection matrices.

Combining this with the pre-image mapping ψ_v learned by Gen-RKM, which can project the feature map $\tilde{\phi}^{(v)}(x^*)$ onto any of the different views, can allow optimization of a combination of objectives defined by the corresponding views. This allows for an " $\mathcal{L}_{\mathcal{V}, \mathcal{AE}}$ attack" like attack as described in Kos, Fischer, and Song (2018), not as the main attack, but as a supplementary attack vector on a different view.

For simplicity we will refer to the pre-image mapping from any latent representation h to the respective view v as $\psi_v(h)$. Note that in the original paper on Gen-RKM the pre-image mapping ψ_v is only taken from the correct feature map $\phi^{(v)}(x^*)$ (Pandey, Schreurs, and Suykens 2021).

The adversarial objective can combine latent-space and multi-view reconstructions into a single loss:

$$\mathcal{L}_{extension} = \alpha_0 L(h^*, h_t) + \sum_v \alpha_v L(\psi_v(h^*), \psi_v(h_t)),$$

where $L(\cdot, \cdot)$ is a generic distance measure, h_t is the target latent code, h^* is the current adversarial latent representation, and the weights α_0, α_v allow balancing contributions from the latent space and different view reconstructions.

Several strategies can be used to optimize this objective:

- **Joint optimization:** All terms in the loss are optimized simultaneously, updating h^* in one unified step.
- **Sequential optimization:** Terms are optimized one after another, e.g., first aligning the latent code, then projecting to each view sequentially.
- **Hybrid or prioritized optimization:** Terms can be weighted or scheduled dynamically, emphasizing certain views or the latent space at different stages of the attack.

In all cases, h^* is directly updated, and the pre-image maps ψ_v are used to obtain the corresponding views reconstructions, allowing a different attack vector to be established.

4.6 Evaluation Metrics

4.6.1 Adversarial Reconstruction Success.

As introduced in Section 4.5.1, we generate adversarial examples $\mathbf{x}^*$ to manipulate the encoder-decoder pipeline. Here, we slightly modify the definition of a successful attack for evaluation purposes.

Previously, success was defined in terms of aligning the reconstruction with the target label y_t . For evaluation, however, we instead compare the reconstruction to the encoding-decoding of the target image itself:

$$\hat{\mathbf{x}}_t = f_{\text{dec}}(f_{\text{enc}}(\mathbf{x}_t)).$$

This change ensures that the success of an attack reflects the quality of reconstructing the target’s latent representation rather than the inherent ability of the model to recognize or represent a class. For instance, a poorly performing model could produce blurry reconstructions that are harder to classify; if we used y_t directly, such reconstructions might be incorrectly counted as unsuccessful, not because the latent-space manipulation failed, but simply because the classifier cannot reliably recognize the blurry output. Making this change is possible and even reasonable due to the assumption of full knowledge of the model by the attacker, the attacker would know what the adversarial reconstruction looked like beforehand and could even use this to alter the optimization in a way they see fit.

To quantify attack success, we use an auxiliary classifier f_{class} and define two complementary criteria that must *both* be satisfied for an attack to be deemed successful:

1. **Target reconstruction alignment:** the adversarial reconstruction matches the reconstruction of the target,

$$f_{\text{class}}(\hat{\mathbf{x}}^*) = f_{\text{class}}(\hat{\mathbf{x}}_t).$$

2. **Source consistency:** the adversarial example itself remains visually close to the source and is classified as such,

$$f_{\text{class}}(\mathbf{x}^*) = y_s.$$

By requiring both criteria, we ensure that an attack is only considered successful if it (i) effectively steers the reconstruction towards the target latent representation and (ii) preserves the perceptual similarity of the adversarial input to the original source.

4.6.2 Mean Squared Error (MSE).

To quantify the distance between two images, we employ the Mean Squared Error (MSE), defined as the average of the squared pixel-wise differences. Given two images $x, \hat{x} \in \mathbb{R}^{H \times W}$ of height H and width W , the MSE is expressed as

$$\text{MSE}(x, \hat{x}) = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W (x_{ij} - \hat{x}_{ij})^2.$$

MSE is a natural choice in our setting, as it directly reflects the objective we optimize during training, making it consistent with the loss function of the model. Although MSE is sensitive to exact pixel values and does not account for perceptual similarity (e.g., two images that look visually similar but are slightly shifted can still yield a high MSE), it provides a simple and effective measure for pixel-level reconstruction fidelity.

Alternative similarity metrics exist that attempt to better capture human perception. For example, the Structural Similarity Index (SSIM) compares images based on luminance, contrast,

and structural information and has been shown to correlate more strongly with subjective visual quality. Although perceptual metrics such as SSIM or more recent neural network-based distances (e.g., LPIPS by R. Zhang et al. (2018)) would be suitable if the aim was to measure how close two images *appear* to a human observer, our focus is on the mathematical reconstruction quality as optimized by the model itself. For this reason, MSE is both the most relevant and consistent evaluation metric in our experiments.

4.6.3 Adversarial Effort (Iterations-to-Success)

To quantify robustness against latent-space attacks, we measure the *adversarial effort* required to achieve a successful attack. Concretely, for a source-target pair $(\mathbf{x}_s, \mathbf{x}_t)$ we define the effort as the number of optimization steps needed until a predefined success criterion is satisfied. We use the following success criterion:

$$\text{success at step } k \iff \left(\|f_{\text{enc}}(\mathbf{x}^{(k)}) - \mathbf{z}_t\|_2 \leq \varepsilon \right) \text{ or } \left(g(\mathbf{x}^{(k)}) = y_t \right),$$

where $\mathbf{z}_t = f_{\text{enc}}(\mathbf{x}_t)$ is the target latent code, $g(\cdot)$ is the downstream classifier, and y_t the target label. The *iterations-to-success* metric is then

$$E(\mathbf{x}_s, \mathbf{x}_t) = \min\{k \in \{1, \dots, K_{\max}\} : \text{success at step } k\}.$$

If no success is observed within $K_{\max}$ iterations, we report $E = +\infty$ (attack failure) or record $K_{\max}$ with a failure flag.

To provide a fair and consistent comparison of robustness across different experiments, we employ a simple gradient descent optimizer with a fixed step size for all attacks. This choice ensures that the update rule remains constant throughout the optimization, so the metric reflects the inherent difficulty of reaching the target (i.e., model robustness) rather than differences in optimizer adaptivity or hardware performance. Unlike adaptive optimizers such as Adam or RMSProp, which modify their step sizes dynamically, fixed-step gradient descent provides a neutral baseline that does not favor one setup over another. Consequently, higher values of E correspond directly to greater robustness of the model under the chosen attack scenario.

4.6.4 Latent Space Alignment (Loss)

In addition to evaluating pixel-space reconstruction quality and adversarial effort, we quantify the alignment in latent space between the current sample and the target. For a sample $\mathbf{x}_{\text{current}}$, we define the latent representation as

$$\mathbf{h}_{\text{current}} = U^\top \phi(\mathbf{x}_{\text{current}}),$$

where $\phi(\cdot)$ denotes the encoder feature mapping and U the projection matrix learned during training. Similarly, the target latent code is

$$\mathbf{h}_{\text{target}} = U^\top \phi(\mathbf{x}_{\text{target}}).$$

The latent space loss is then computed as the Euclidean distance between these vectors:

$$\mathcal{L}_{\text{latent}}(\mathbf{x}_{\text{current}}, \mathbf{x}_{\text{target}}) = \|\mathbf{h}_{\text{current}} - \mathbf{h}_{\text{target}}\|_2.$$

This metric directly measures how closely the current sample aligns with the target in the model’s internal representation. While MSE captures visual similarity, the latent loss reflects how effectively the model’s internal encoding has been matched.

During the optimization or adversarial procedure, $\mathbf{x}_{\text{current}}$ is iteratively updated to minimize the latent loss. Computing $\mathcal{L}_{\text{latent}}$ at each iteration allows us to monitor convergence dynamics and assess the difficulty of steering the sample in latent space, complementing both pixel-space metrics (MSE) and the iteration-based robustness measure.

Change for autoencoder Models The same latent space alignment metric is applied to autoencoder-based models, where the encoder provides the latent embedding $\mathbf{z}(\cdot)$ of the source image $\mathbf{x}_{\text{source}}$ and the target image $\mathbf{x}_{\text{target}}$. The adversarial example $\mathbf{x}^*$ is then iteratively updated to minimize the Euclidean distance between its latent vector and that of the target:

$$\mathcal{L}_{\text{latent}} = \|\mathbf{z}(\mathbf{x}^*) - \mathbf{z}(\mathbf{x}_{\text{target}})\|_2.$$

4.7 Robustness Evaluation Framework

To ensure comparability across models and attack types, we define a unified robustness evaluation framework. This framework specifies how adversarial examples are generated, how the models are queried or reconstructed, and which metrics are computed. In practice, the evaluation proceeds as follows: (i) select a clean input, (ii) generate an adversarial counterpart using the latent attack framework (Sec. 4.5), (iii) reconstruct the output from the model under attack, and (iv) compute the robustness metrics (Sec. 4.6).

The framework is applied consistently across all models under study (autoencoders and Gen-RKM), with only minor adaptations where necessary. This ensures a fair comparison of robustness under identical threat models and evaluation settings. Ultimately, it serves as the link between the attack methodology and the experimental results.

4.7.1 Multi-View Robustness Evaluation (Gen-RKM Extension)

In the standard latent attack described in Section 4.5.1, the adversary perturbs the latent representation of the model, steering it towards a target. In the setting of multi-view representation learning, as in Gen-RKM, this corresponds to manipulating the *shared latent representation* $h \in \mathcal{H}$, which simultaneously influences all views, even though only a single view may be optimized during the attack.

In Gen-RKM, the shared latent representation h serves as a common source from which different views are reconstructed using learned pre-image mappings (neural networks). For example, in the MNIST case, one can reconstruct both the *image view* $g_{\text{img}}(h)$ and the *label view* $g_{\text{lbl}}(h)$. Importantly, these mappings themselves are neural networks, meaning that in a full white-box setting they could also be exploited by an adversary. Nevertheless, their existence enables us to define a more stringent evaluation of robustness in the multi-view setting.

Given a perturbed input $\mathbf{x}^*$ with shared latent representation $h^* = f_{\text{enc}}(\mathbf{x}^*)$, we extend the evaluation metrics from Section 4.6 by introducing a *cross-view consistency criterion*. An attack is deemed successful only if:

1. **Adversarial reconstruction success** the adversarial reconstruction is a success as defined in Section 4.6 and has both *target reconstruction alignment* and *source consistency*.
2. **Cross-view consistency:** if another view explicitly encodes the target class (e.g., the label view), then it must agree with the classification of the reconstruction,

$$g_{\text{lbl}}(h^*) = f_{\text{class}}(g_{\text{img}}(h^*)).$$

This extension ensures that adversarial success is not only measured by fooling a single reconstruction but also by maintaining consistency across multiple generated views of the same latent representation. In this way, the success metric more closely reflects the robustness of a system that leverages multi-view representation as an implicit defense mechanism.

5 Experiments

We evaluate our models and attack strategies on the datasets introduced in Section 4.2. Performance is reported using the evaluation metrics defined in Section 4.6.

5.1 MNIST Source and Target Image Selection

For the evaluation of our models, we constructed a subset of the MNIST dataset specifically to form clear **source-target** pairs. Rather than using the last 10,000 images from the training set as a validation set, we utilized the official MNIST test set (10,000 images) to perform final evaluations, ensuring that no images used during training or hyperparameter tuning are included in this evaluation. This approach guarantees that our results reflect true generalization to unseen data.

From this test set, we selected 5 images per digit class to serve as **source** examples, and another 5 images per class to serve as **target** examples. Source and target images are disjoint, meaning no image appears in both sets. This selection results in a total of 100 images (50 sources and 50 targets).

To systematically create these pairs, we first grouped all images by their digit label and shuffled the indices within each class for reproducibility. We then sampled the first 5 images per class as sources, and the next 5 images per class as targets. For cross-class evaluation, we generated all possible source-target pairs with differing labels, resulting in 450 pairs when using 5 sources and 5 targets per class. This pairing allows us to quantitatively and qualitatively evaluate the model’s ability to map between digits of different classes.

Figure 4 visualizes the selected images. The left panel shows the source images for each digit class, while the right panel shows the corresponding target images. The structured selection of sources and targets provides a clear framework for evaluating how well the models preserves digit identity and transforms features between classes.

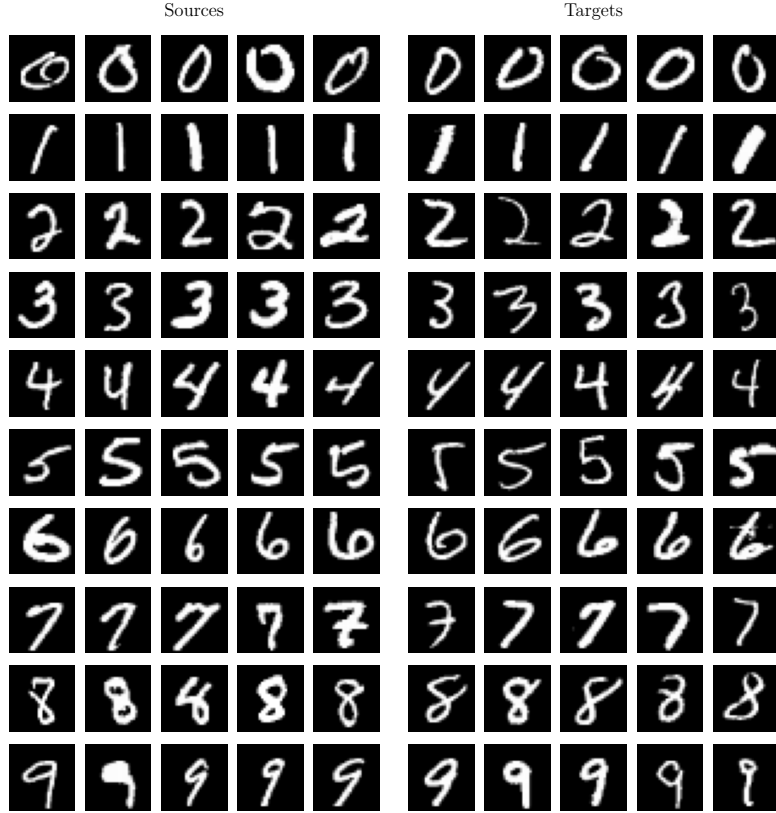


Figure 4: Visualization of the source (left) and target (right) images used in the experiments. Each row corresponds to a different digit class (0–9), and each column shows one of the 5 selected images per class.

5.2 Experiment 1: Robustness to Latent-Space Adversarial Attacks

5.2.1 Setup

In this experiment, we evaluated the robustness of four generative models on the MNIST dataset: VAE, β -VAE, β -TC-VAE, and Gen-RKM. All experiments were conducted on the MNIST test set that was held out from training. For the autoencoder-based models (VAE variants), each model was trained for 20 epochs, whereas for Gen-RKM we used the pretrained model as described in Section 3.3.4.

The latent-space adversarial attack was applied as described in Section 4.5.1. We used the Adam optimizer with a learning rate of 0.01 for a maximum of 100 steps, employing early stopping according to the criterion defined in the methodology. Since Adam automatically adjusts the step size, the total number of optimization steps is not reported as a metric for this experiment. The distance measure $L(\cdot, \cdot)$ used to compute the latent-space loss is the L2 norm, and the early stopping criterion was implemented using the pre-trained LeNet classifier introduced in Section 3.3.5.

Source-target image pairs were selected as described in Section 5.1. For each attack, we measured the metrics introduced in Section 4.6, namely the attack success rate based on the reconstructed adversarial example, the mean squared error (MSE) between the source image

and the adversarial example, and the decrease in latent-space loss from the start to the end of the attack. The MSE and the latent-space loss difference are reported only for successful attacks, as these metrics are meaningful primarily when the attack successfully changes the reconstructed label. The average decrease in latent-space loss provides insight into how much the latent representation is altered during successful attacks. It is also worth noting that the attack success metric mentioned is the basic attack metric from 4.6 and has not yet been extended with cross-view consistency as mentioned in 4.7.1 to provide a consistent metric across models. All experiments were performed on a CUDA-enabled GPU, ensuring consistent and efficient gradient-based optimization across all models.

5.2.2 Results

Table 2 shows the robustness of all evaluated models against latent-space adversarial attacks on MNIST. The table includes the attack success rate (ASR), the average mean squared error (MSE) of successful attacks, and the average decrease in latent-space loss for successful attacks. Table 8 of the Appendix shows extra information on how different models performed depending on what class was being attacked, showing that some classes are significantly harder to attack than others, especially for VAE models.

The results indicate that Gen-RKM is the least robust model, with the highest ASR of 96.44%, meaning adversarial attacks succeed more frequently. In contrast, the VAE-based models (β -VAE, β -TC-VAE, and VAE) exhibit lower ASRs, ranging from 87.78% to 89.11%, indicating higher robustness. Among these, β -TC-VAE achieves the highest ASR (89.11%), followed closely by β -VAE and VAE.

The average MSE is also lowest for Gen-RKM, reflecting that attacks require smaller perturbations, this combined with the high ASR emphasizes that it is easier to attack successfully, as it needs a smaller amount of perturbation on the input and still leads to a higher ASR. The latent-space loss decrease shows a similar pattern: successful attacks on Gen-RKM require almost no latent-space modification, while attacks on VAE variants involve larger latent-space adjustments.

Table 2: Overall robustness of different models against latent-space adversarial attacks on MNIST. Avg Loss Diff indicates the average decrease in latent space loss for successful attacks.

Model	Attack Success Rate (%)	Average MSE	Avg Loss Diff
VAE	87.78	0.0357	2.00
β -VAE	87.78	0.0346	1.30
β -TC-VAE	89.11	0.0348	1.65
Gen-RKM	96.44	0.0110	0.007

5.3 Experiment 2: Cross-view consistency for multi-view latent targeting of Gen-RKM

5.3.1 Setup

This experiment evaluates the robustness of the Gen-RKM model under the proposed multi-view latent attack framework described in 4.5.4 while measuring the attack success as described in 4.7.1. The primary objective is to test whether cross-view consistency can act as a defense mechanism by exposing adversarial perturbations that are optimized to fool only a single view, while failing to maintain consistency across multiple reconstructed views of the shared latent representation h . In addition, we also study an extension of this framework in which the attacker

directly targets the latent feature representations of each view. This allows us to investigate whether adversarial success can be amplified or diminished when explicitly balancing the influence of different views in the attack objective.

The model under evaluation is the Gen-RKM, trained as in the baseline setting, but augmented with a latent loss term that balances discrepancies across view-specific latent representations as mentioned in 4.5.4. This loss is weighted by hyperparameters that control the relative emphasis on each view, thereby creating multiple attack scenarios of varying difficulty. The adversary is allowed to attack under different configurations of these weights, attempting to simultaneously mislead both the reconstruction and the label prediction.

We evaluate the framework on the MNIST dataset, where the two considered views are: (a) the input image, reconstructed via a learned pre-image mapping from the latent space, and (b) the digit label, reconstructed through a projection matrix followed by a neural network pre-image mapping. Attacks are performed by perturbing the image representation x^* with the goal of producing both incorrect reconstructions and inconsistent label predictions. The source and target image selection follows the setup of Section 5.1 and is in line with the experiment in Section 5.2.

Evaluation criteria follow the extended success metrics defined in Section 4.7.1. Specifically, adversarial success is measured not only in terms of view-specific attack performance (e.g., reconstruction quality degradation and label misclassification), but also in terms of cross-view consistency, which quantifies whether both views remain aligned under adversarial perturbation. Together, these metrics allow us to assess both the direct effectiveness of the attack and the potential of cross-view consistency to act as a built-in robustness mechanism.

Formally, the adversarial perturbation is generated by optimizing a weighted loss function that balances different objectives. Let h^* denote the adversarial latent code, obtained by perturbing the adversarial example x^* . The loss consists of three terms:

$$\mathcal{L}(h^*) = \alpha \|h^* - h_t\|_2^2 + \beta \|g_A(h^*) - x_t\|_2^2 + \gamma \mathcal{L}_{CE}(g_B(h^*), y_t),$$

where each component corresponds to:

- (i) $\|h^* - h_t\|_2^2$ encourages the perturbed latent code to move towards the target latent representation h_t ,
- (ii) $\|g_A(h^*) - x_t\|_2^2$ enforces that the adversarial reconstruction of view A (the image) aligns with the target x_t ,
- (iii) $\mathcal{L}_{CE}(g_B(h^*), y_t)$ enforces that the adversarially generated label of view B matches the target label y_t .

Here $\alpha, \beta, \gamma \in \mathbb{R}_{\geq 0}$ are hyperparameters controlling the relative weight of each term, $g_A(\cdot)$ denotes the learned pre-image mapping for the image view, and $g_B(\cdot)$ denotes the projection-network mapping for the label view. The generic $L(\cdot, \cdot)$ distance measure is filled in by $\|\dots\|_2^2$ representing the L2 norm and $\mathcal{L}_{CE}$ representing the cross-entropy, these are used for the image and the label view respectively.

We use the Adam optimizer with a learning rate of 0.01 for a maximum of 100 steps to optimize this loss function.

To evaluate the effectiveness of attacks, we report three distinct success criteria:

1. **Source-consistency success:** the adversarial perturbation must mislead the model with respect to the original reconstruction of the source view.

2. **View A (image) success:** the adversarial reconstruction $g_A(h^*)$ aligns with the targeted image x_t when evaluated by the LeNet classifier defined in Section 4.3.3.
3. **View B (label) success:** the adversarial label prediction $g_B(h^*)$ matches the target class y_t as predicted by LeNet on the target image.

An attack is considered *fully successful under the cross-view defense* if and only if all three criteria are satisfied simultaneously. This stricter definition ensures that adversarial success reflects consistent manipulation of the shared latent representation rather than exploiting weaknesses in a single view.

The stopping criterion is also altered to accommodate for the changes in success metric. The attack will only stop early when both of the views have achieved reconstruction success, this way we allow the algorithm to still change a minimal amount of pixels to achieve full success as defined in this experiment.

We evaluate the model across a range of different values for α, β, γ , thereby gaining insight into how the balance between latent alignment, image reconstruction, and label prediction affects both adversarial success and the robustness conferred by cross-view consistency.

5.3.2 Results

For the following tables the *Source Cons.* indicates the source consistency, which is one of the necessary components for a successful attack, it points out whether or not the adversarial example is still recognizable as the source class before reconstruction. *View 1* and *View 2* point to the image reconstruction success and the label reconstruction success respectively as defined in the setup.

Table 3: Results when a single loss weight dominates (100% in one objective). All values are expressed in percent.

α	β	γ	Source Cons.	View 1	View 2	Total
100	0	0	98.67	98.00	10.00	9.78
0	100	0	98.22	97.56	10.00	9.33
0	0	100	89.11	27.56	99.78	26.00

Dominant loss settings. Table 3 shows the performance when one of the loss terms dominates (i.e., one hyperparameter set to 100% while the others are zero). We observe that when only the latent loss or view 1 loss is active (α or β is one), the model performs well on both source consistency and the reconstruction of view 1 (around 98%), but fails to generalize to view 2, leading to very low overall success (below 10%). In contrast, when only the view 2 loss is active, that view achieves almost perfect performance ($\approx 100\%$), but both source consistency and mainly view 1 success degrade sharply, resulting in a low total success rate, but still outperforming the other two instances. This highlights the necessity of balancing the different objectives rather than optimizing a single loss in isolation when a detection mechanism is present. Notably, the first case in Table 3, with $\alpha = 100\%$, uses the same loss function as in Experiment 5.2. However, due to differences in the success metric and stopping criteria, this setting produces different results.

Table 4: Results under general trade-off settings between losses. All values are expressed in percent. Ordered by ascending total success.

α	β	γ	Source Cons.	View 1	View 2	Total
0	75	25	97.33	97.78	10.00	9.56
33	33	33	97.11	97.78	10.00	9.56
0	50	50	97.11	98.00	10.00	9.78
25	0	75	88.44	29.56	100.00	28.00
0	25	75	97.56	97.78	30.44	29.11
50	0	50	88.22	33.56	100.00	32.22

General trade-off settings. The results of intermediate trade-offs are summarized in Table 4, ordered by ascending total success. We see that purely balanced configurations (e.g., $\alpha = \beta = \gamma = 33\%$) still fail to improve the overall success rate, which remains below 10%. Slightly skewed settings, such as giving more weight to γ (view 2 loss), lead to moderate improvements (e.g., 28–32% total success). Nevertheless, none of these general trade-offs achieve a satisfactory balance: success remains dominated by performance on either view 1 or view 2, but not both. This suggests that coarse weighting alone is insufficient, and finer tuning is required.

Table 5: Results with refined hyperparameter tuning for balanced losses. All values are expressed in percent. Ordered by ascending total success.

α	β	γ	Source Cons.	View 1	View 2	Total
2.5	0	97.5	87.56	26.44	99.56	25.11
10	0	90	87.56	27.78	100.00	26.00
0	5	95	97.33	96.00	70.67	65.78
0	2.5	97.5	94.22	90.67	89.33	76.89
10	2.5	87.5	95.33	91.56	87.78	77.11
10	2	88	94.44	89.33	90.89	77.33

Final tuned settings. The results of refined hyperparameter tuning are presented in Table 5, ordered by ascending total success. Here, we see a marked improvement compared to the previous settings: combinations such as $\alpha = 10\%$, $\beta = 2\%$, $\gamma = 88\%$ yield a total success rate above 77%. This demonstrates that carefully balancing the losses with a strong emphasis on the view 2 term, but not exclusively, is key to achieving good performance across all three metrics simultaneously. In particular, the best-performing settings maintain high view 2 accuracy ($\approx 90\%$) without sacrificing consistency and view 1 performance, leading to the highest total success in this experiment.

Summary. Across the different experimental conditions, we find that no single loss term can drive good overall performance on its own: while source consistency and view 1 achieve near-perfect accuracy in isolation, view 2 collapses, and vice versa. Naïve trade-offs also fail to improve the total success rate substantially. Only after fine-grained tuning do we obtain a balanced configuration that preserves high accuracy across all three metrics simultaneously, pushing the total success rate beyond 77%. This underlines the importance of multi-view capabilities, where

a careful balance in view specific losses is needed to obtain a consistently successful attack when a cross-view defensive measure is integrated.

5.4 Discussion

The experiments highlight several key insights about the robustness of generative models under latent-space adversarial attacks and the role of multi-view consistency as a defensive mechanism.

First, Experiment 1 demonstrates that Gen-RKM, while powerful in representation, is more susceptible to latent-space adversarial perturbations compared to VAE-based models. Its high attack success rate (96.44%) combined with minimal perturbation required underscores the sensitivity of the latent representations learned by Gen-RKM. In contrast, VAE variants exhibit lower attack success rates and require larger modifications in latent space, indicating relatively stronger robustness. This suggests that latent-space flexibility, while beneficial for generative performance, may come at the cost of vulnerability to targeted attacks.

Experiment 2 extends these findings by evaluating cross-view consistency in multi-view Gen-RKM. The results show that attacks targeting only a single view or a single loss component fail to generalize across views, as reflected in the low total success rates when one hyperparameter dominates. Naïve trade-offs improve overall success modestly, but only careful tuning of the latent, view1, and view 2 loss weights achieves a balanced configuration, reaching above 77% total success. These findings highlight that multi-view constraints effectively act as a defense: adversarial perturbations must satisfy consistency across multiple outputs, which increases the difficulty of successful attacks.

Taken together, the experiments reveal a clear trade-off between attack complexity and model robustness. Gen-RKM’s latent representations are highly vulnerable in a single-view context, but the introduction of cross-view consistency imposes structural constraints that enhance robustness. This structural advantage is reflected in Table 6: the attack success rate for Gen-RKM drops from 96.44% to 77.33% when multi-view consistency is enforced. Remarkably, this not only reduces vulnerability but actually positions Gen-RKM as more robust than the VAE-based models (87.78–89.11%) under the same conditions. Multi-view constraints therefore not only defend against attacks but also leverage the model’s latent structure to achieve superior robustness compared to alternative generative approaches.

Table 6: Attack success rate of different models against latent-space adversarial attacks on MNIST expanded with the success rate of Gen-RKM when multi-view consistency is enforced.

Model	Attack Success Rate (%)
Gen-RKM (extra defense)	77.33
VAE	87.78
β -VAE	87.78
β -TC-VAE	89.11
Gen-RKM	96.44

Overall, these observations provide both a benchmark for latent-space robustness and a methodological framework for future studies on multi-view generative models, adversarial attacks, and defense strategies. The results underscore the importance of multi-view modeling: only by carefully balancing view-specific losses can one achieve high performance across all evaluation metrics simultaneously. In the following section, we further analyze these results, comparing attack effectiveness and defense performance across different hyperparameter settings, and visualize the trade-offs between source consistency, image reconstruction, and label reconstruction.

6 Results and Discussion

This section presents the results obtained from Section 5 and discusses their implications. The central goal of this thesis was to investigate the link between disentanglement and robustness in a generative learning setting, with particular emphasis on multi-view representation learning. To this end, we examined the Gen-RKM framework as a multi-view generative model capable of producing multiple views from a shared latent representation. We contrasted this model against a range of disentanglement-focused autoencoders that were also used as baselines in the original Gen-RKM work (Pandey, Schreurs, and Suykens 2021).

In Section 4.4, we confirmed based on Pandey, Schreurs, and Suykens (2021) that Gen-RKM achieves the highest degree of disentanglement, followed by β -TC-VAE, β -VAE, and VAE, especially when the latent dimensionality is close to the true number of generative factors as is the case for MNIST and the pre-trained models considered here.

To evaluate robustness, we implemented the $\mathcal{L}_{latent}$ attack adapted from Kos, Fischer, and Song (2018). This attack perturbs MNIST inputs at the pixel level to produce adversarial reconstructions that resemble a different target digit. For evaluation, 450 attack instances were generated by pairing five source and five target samples for each digit class (see Section 5.1). An attack was considered successful if (i) the adversarial example was still classified by a LeNet classifier as the same class as the source image, while (ii) its reconstruction was classified as the target digit.

6.1 Experiment 1: Disentanglement vs. Robustness

Experiment 1 (Section 5.2) revealed a surprising trend: more disentangled models, contrary to claims in prior work (Alemi et al. 2016; Pandey, Schreurs, and Suykens 2021), were less robust to adversarial perturbations. Both attack success rates and the magnitude of required perturbations suggest that disentangled autoencoders are more vulnerable. Several factors may explain this. First, disentangled models encode semantically meaningful factors of variation in a structured way, making it easier for an adversary to manipulate only the components necessary for a targeted attack. Second, the lower robustness of VAEs may stem partly from their inherently blurrier and noisier reconstructions, which sometimes cause natural misclassifications and thereby inadvertently reduce adversarial effectiveness. Finally, Gen-RKM’s multi-view latent representation, which encodes both the image and its label, introduces an additional vulnerability: an attacker can modify label-related components in the latent space without substantially altering the image appearance, this is actually likely even more effective due to the disentanglement of the model.

6.2 Experiment 2: Leveraging Multi-View Representations as a Defense

In Experiment 2 we explicitly exploited this multi-view structure to build a defense mechanism. By enforcing consistency between the reconstructed image and its associated label representation, adversarial samples could be detected. Redefining the success metric to require alignment between image, label, and latent representation caused a drastic drop in attack success: from 96.44% without defense to just 9.78% under the new criterion.

We then extended the attack with an adapted loss function (Section 4.5.4) that jointly targeted the latent vector, the image view, and the label view, controlled by hyperparameters α , β , and γ . Results showed that:

- Attacking a single view led to low success rates (10–26%).

- Moderate tuning raised success to 32%.
- Only careful balancing across all views increased success to 77%.

Despite this, Table 6 demonstrates that Gen-RKM with the detection-based defense remained more robust than even the simplest VAE baseline, with 10% lower attack success. This highlights that Gen-RKM’s multi-view structure, when leveraged defensively, can surpass standard autoencoders in robustness.

6.3 Discussion of Loss Balancing and Trade-offs

The tuned hyperparameters may appear arbitrary at first, but reflect the fundamentally different loss formulations. The latent and image-view losses use an L2 norm, though on distinct representations (latent vectors vs. pixel values), while the label-view loss is based on cross-entropy over logits. This creates a balancing act: adversarial optimization tends to focus primarily on the image view, since pixel-level modifications directly optimize for visual misclassification. The latent loss, while formally covering all views, still favors image consistency because of this bias.

Taken together, Experiment 1 demonstrates that Gen-RKM’s complexity and multi-view latent space introduce vulnerabilities not present in simpler models. However, Experiment 2 shows that the same structure can be harnessed to drastically improve robustness when used as part of a targeted defense. This duality highlights both the risks and opportunities of multi-view generative modeling in adversarial settings.

6.4 Linking back to the thesis question

The central research question of this thesis asked whether disentanglement improves robustness in generative representation learning, particularly in multi-view models. The experiments demonstrate that disentanglement alone does not guarantee robustness: the VAE variants all achieve broadly similar attack success rates, and Gen-RKM without defense is notably more vulnerable, reaching over 96% attack success. However, when cross-view consistency is exploited as part of a targeted defense, Gen-RKM’s robustness improves substantially, outperforming all VAE baselines. This finding highlights that robustness is not a direct byproduct of disentanglement, but can emerge when disentangled or structured representations are actively leveraged in defense mechanisms.

6.5 Broader implications

These results carry several broader implications. First, they caution against assuming that improved disentanglement automatically translates into better security in generative systems. Second, they suggest a promising research direction: multi-view generative models like Gen-RKM can integrate disentangled structure not only for interpretability, but also as a practical tool for adversarial defense. By enforcing consistency across views (e.g., image vs. label), such models introduce an additional barrier for adversaries that does not exist in single-view disentangled autoencoders. In this sense, disentanglement and robustness are not inherently aligned, but their interplay can be purposefully designed to make generative learning systems more secure and reliable.

7 Conclusion

This thesis set out to investigate whether disentanglement improves robustness in generative representation learning, particularly under latent-space adversarial attacks and in multi-view generative models.

The results demonstrate that disentanglement alone does not inherently improve robustness. On the contrary, models with stronger representational power and disentangled structure, such as Gen-RKM, proved more vulnerable to targeted latent-space perturbations than VAE-based counterparts. However, when multi-view consistency was introduced as an explicit defense mechanism, Gen-RKM’s robustness improved substantially, lowering attack success rates by nearly 20% compared to its single-view setting. This highlights a key insight: disentanglement is not automatically protective, but it can be strategically exploited to design effective defenses when paired with structural constraints such as cross-view consistency.

Beyond this main finding, the experiments underscore broader lessons for the design of generative systems. Robustness cannot be assumed from disentanglement or representation learning objectives alone. Instead, robustness arises when structural properties of models are deliberately leveraged against adversaries. Multi-view architectures, in particular, provide natural opportunities for enforcing consistency constraints that increase attack difficulty, making them promising candidates for adversarially robust generative modeling.

This work is not without limitations. The evaluation focused on MNIST, a relatively simple dataset, and on specific families of generative models. Broader validation on more complex data, such as natural images or multimodal datasets, will be necessary to assess the generality of these findings. Moreover, adversarial strategies were limited to latent-space perturbations; exploring other attack surfaces may further refine our understanding of model vulnerabilities. Finally, disentanglement was not reevaluated under the most recent frameworks, but instead relied on results from an established paper. A systematic re-examination of disentanglement with up-to-date methods therefore remains an important task for future work.

Future research should therefore expand in three directions. First, scaling experiments to richer datasets and more advanced generative models will test whether the benefits of cross-view consistency generalize. Second, directly reevaluating disentanglement within modern frameworks will clarify its role in shaping robustness, and potentially uncover new interactions with adversarial defenses. Third, exploring combinations of disentanglement with complementary defense mechanisms may yield models that are not only interpretable but also resilient. In this way, the interplay between disentanglement, multi-view consistency, and adversarial robustness offers a promising frontier for both theoretical understanding and practical applications of generative modeling.

In summary, disentanglement by itself does not ensure robustness. Yet, when paired with multi-view consistency, it becomes a powerful tool for designing defenses. This thesis thus contributes both a cautionary note against simplistic assumptions about disentanglement and a constructive path forward: leveraging structure in generative models not only for interpretability, but also for security and reliability.

A LeNet Classification Metrics

A.1 Validation Accuracy and loss per Epoch

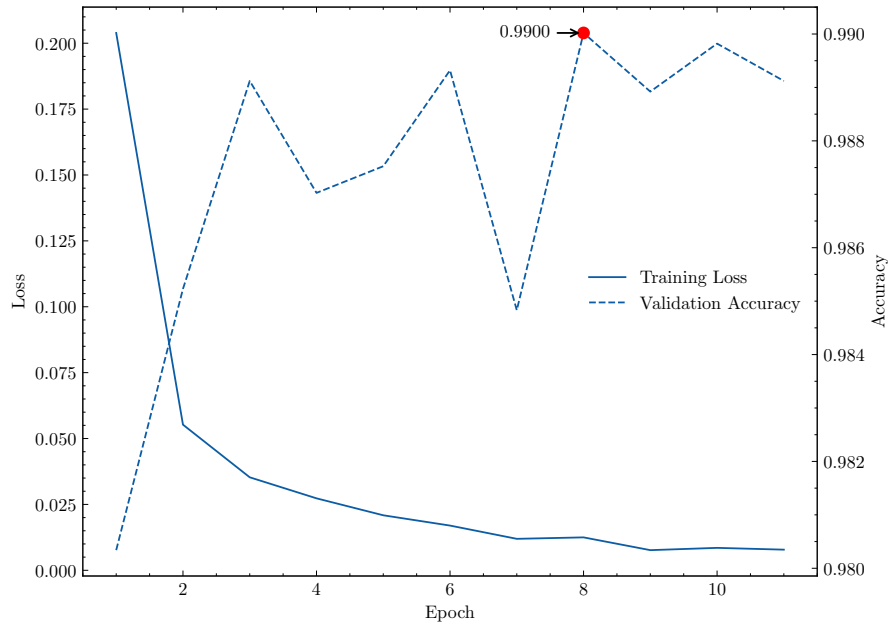


Figure 5: LeNet validation accuracy and loss during training on MNIST. Early stopping was applied after 11 epochs and the model of epoch number 8 was saved and used.

A.2 Classification Report

Table 7: LeNet per-class performance on the MNIST test set. Overall test accuracy: **98.94%**.

Class	Precision	Recall	F1-score	Support
0	0.989	0.996	0.992	980
1	0.998	0.987	0.993	1135
2	0.983	0.992	0.988	1032
3	0.983	0.999	0.991	1010
4	0.997	0.982	0.989	982
5	0.982	0.991	0.987	892
6	0.995	0.983	0.989	958
7	0.991	0.985	0.988	1028
8	0.994	0.990	0.992	974
9	0.981	0.989	0.985	1009
Accuracy	0.989			
Macro Avg	0.989	0.989	0.989	10000
Weighted Avg	0.990	0.989	0.989	10000

B Validation Loss of the Autoencoders

This appendix shows the validation loss curves for all autoencoder models used in our experiments. These plots provide an overview of the training dynamics and convergence behavior but are not directly used in the robustness evaluation. All autoencoders showed signs of convergence at the 20 epoch mark where we cut off training.

B.1 VAE

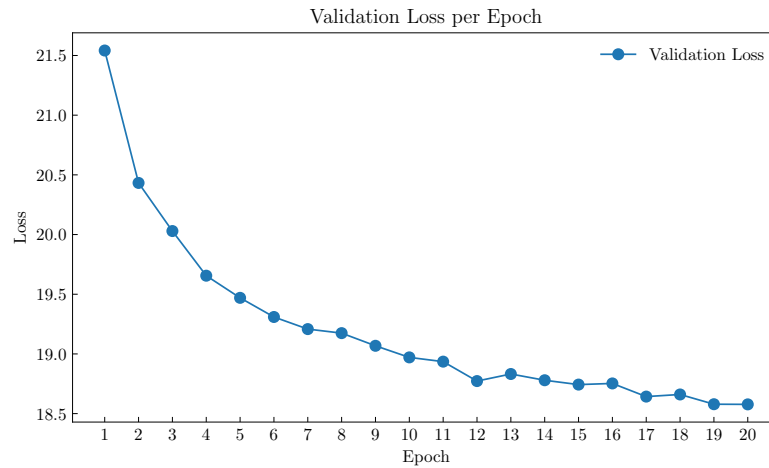


Figure 6: Validation loss curve of the standard Variational Autoencoder (VAE) over epochs. The loss steadily decreases, indicating stable training convergence.

B.2 β -VAE

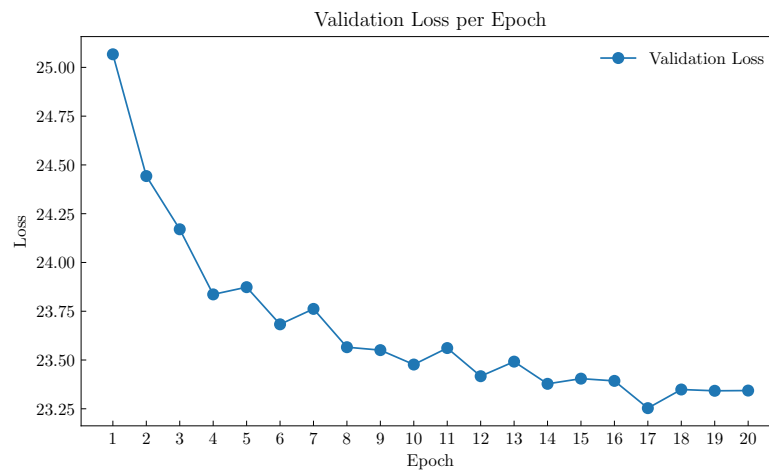


Figure 7: Validation loss curve of the β -VAE over epochs. The loss shows stable convergence while enforcing increased latent disentanglement through the β parameter.

B.3 β -TC-VAE

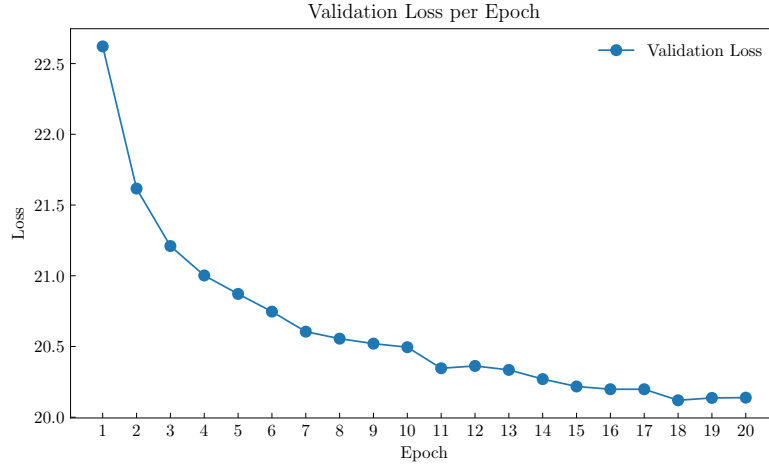


Figure 8: Validation loss curve of the β -TC-VAE over epochs. The curve reflects stable training while promoting total correlation penalization in the latent space.

C Additional Information on Experiment 1

Table 8: Per-class attack success rate (ASR) of different models.

Model	0	1	2	3	4	5	6	7	8	9
VAE	64.44	97.78	95.56	91.11	97.78	88.89	64.44	86.67	97.78	93.33
β -VAE	71.11	97.78	93.33	95.56	95.56	95.56	60.00	82.22	97.78	88.89
β -TC-VAE	71.11	100.00	93.33	100.00	91.11	95.56	62.22	88.89	100.00	88.89
Gen-RKM	97.78	95.56	95.56	97.78	95.56	97.78	91.11	97.78	97.78	97.78

References

- Abdollahpoorrostam, Alireza, Mahed Abroshan, and Seyed-Mohsen Moosavi-Dezfooli (Dec. 2024). “Re-visiting DeepFool: generalization and improvement”. DOI: 10.48550/arXiv.2303.12481.
- Alemi, Alexander A. et al. (2016). “Deep Variational Information Bottleneck”. arXiv: 1612.00410.
- Almudévar, Antonio et al. (Aug. 2024). “Defining and Measuring Disentanglement for non-Independent Factors of Variation”. DOI: 10.48550/arXiv.2408.07016.
- Carboneau, Marc-André et al. (July 2024). “Measuring Disentanglement: A Review of Metrics”. In: *IEEE Transactions on Neural Networks and Learning Systems* 35.7, pp. 8747–8761. DOI: 10.1109/TNNLS.2022.3218982.
- Carlini, Nicholas, Anish Athalye, et al. (Feb. 2019). “On Evaluating Adversarial Robustness”. DOI: 10.48550/arXiv.1902.06705.
- Carlini, Nicholas and David Wagner (May 2017). “Towards Evaluating the Robustness of Neural Networks”. In: *2017 IEEE Symposium on Security and Privacy (SP)*, pp. 39–57. DOI: 10.1109/SP.2017.49.
- Chadebec, Clément (2022). “benchmark VAE. software implementation”. Accessed: 25-08-2025. URL: https://github.com/clementchadebec/benchmark_VAE/tree/main/examples/notebooks/models_training.
- Chadebec, Clément, Louis Vincent, and Stephanie Allasonniere (Dec. 2022). “Pythae: Unifying Generative Autoencoders in Python - A Benchmarking Use Case”. In: *Advances in Neural Information Processing Systems* 35, pp. 21575–21589. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/872f0e04ef95be7970d9a9d74b198fdf-Abstract-Datasets_and_Benchmarks.html.
- Chakraborty, Anirban et al. (2021). “A survey on adversarial attacks and defences”. In: *CAAI Transactions on Intelligence Technology* 6.1, pp. 25–45.
- Chen, Ricky T. Q. et al. (2018). “Isolating Sources of Disentanglement in Variational Autoencoders”. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc. URL: <https://proceedings.neurips.cc/paper/2018/hash/1ee3dfcd8a0645a25a35977997223d22-Abstract.html>.
- Cheung, Brian et al. (June 2015). “Discovering Hidden Factors of Variation in Deep Networks”. DOI: 10.48550/arXiv.1412.6583.
- Dalvi, Nilesh et al. (2004). “Adversarial classification”. In: *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 99–108.
- Desjardins, Guillaume, Aaron Courville, and Yoshua Bengio (Oct. 2012). “Disentangling Factors of Variation via Generative Entangling”. DOI: 10.48550/arXiv.1210.5474.
- Eastwood, Cian and Christopher K I Williams (May 2018). “A Framework for the Quantitative Evaluation of Disentangled Representations”. In: *Sixth International Conference on Learning Representations (ICLR 2018)*.
- Gondal, Muhammad Waleed et al. (2019). “On the transfer of inductive bias from simulation to the real world: a new disentanglement dataset”. In: *Advances in Neural Information Processing Systems* 32, pp. 15740–15751.

- Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy (Mar. 2015). “Explaining and Harnessing Adversarial Examples”. DOI: 10.48550/arXiv.1412.6572.
- Higgins, Irina, David Amos, et al. (Dec. 2018). “Towards a Definition of Disentangled Representations”. DOI: 10.48550/arXiv.1812.02230.
- Higgins, Irina, Loic Matthey, et al. (2017). “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Sy2fzU9gl>.
- Kingma, Diederik and Jimmy Ba (Dec. 2014). “Adam: A Method for Stochastic Optimization”. DOI: 10.48550/arXiv.1412.6980.
- Kingma, Diederik P. and Max Welling (2013). “Auto-Encoding Variational Bayes”. DOI: 10.48550/arXiv.1312.6114.
- Kos, Jernej, Ian Fischer, and Dawn Song (2018). “Adversarial Examples for Generative Models”. In: *2018 IEEE Security and Privacy Workshops (SPW)*, pp. 36–42. DOI: 10.1109/SPW.2018.00014.
- Lecun, Y. et al. (Nov. 1998). “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324. DOI: 10.1109/5.726791.
- Liu, Ziwei et al. (Dec. 2015). “Deep Learning Face Attributes in the Wild”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 3730–3738. URL: https://openaccess.the-cvf.com/content_iccv_2015/html/Liu_Deep_Learning_Face_ICCV_2015_paper.html.
- Moosavi-Dezfooli, Seyed-Mohsen, Alhussein Fawzi, and Pascal Frossard (June 2016). “DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2574–2582. URL: https://openaccess.the-cvf.com/content_cvpr_2016/html/Moosavi-Dezfooli_DeepFool_A_Simple_CVPR_2016_paper.html.
- Pandey, Arun, Joachim Schreurs, and Johan A. K. Suykens (Mar. 2021). “Generative Restricted Kernel Machines: A framework for multi-view generation and disentangled feature learning”. In: *Neural Networks* 135, pp. 177–191. DOI: 10.1016/j.neunet.2020.12.010.
- (2021b). “Gen-RKM: Generative Restricted Kernel Machines - code”. Accessed: 25-08-2025. URL: <https://www.esat.kuleuven.be/stadius/E/pandey/softwareGenRKM.php>.
- Ridgeway, Karl and Michael C Mozer (2018). “Learning Deep Disentangled Embeddings With the F-Statistic Loss”. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc. URL: <https://proceedings.neurips.cc/paper/2018/hash/2b24d495052a8ce66358eb576b8912c8-Abstract.html>.
- Suykens, Johan A. K. (Aug. 2017). “Deep Restricted Kernel Machines Using Conjugate Feature Duality”. In: *Neural Computation* 29.8, pp. 2123–2163. DOI: 10.1162/neco.a.00984.
- Szegedy, Christian et al. (Feb. 2014). “Intriguing properties of neural networks”. DOI: 10.48550/arXiv.1312.6199.
- Toderici, George et al. (July 2017). “Full Resolution Image Compression With Recurrent Neural Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5306–5314. arXiv: 1608.05148.
- Xu, Chang, Dacheng Tao, and Chao Xu (2013). “A Survey on Multi-view Learning”. arXiv: 1304.5634.

Zhang, Quan-shi and Song-Chun Zhu (2018). “Visual interpretability for deep learning: a survey”. In: *Frontiers of Information Technology & Electronic Engineering* 19.1, pp. 27–39. DOI: 10.1631/FITEE.1700808.

Zhang, Richard et al. (2018). “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. arXiv: 1801.03924.